

计算机硬件从零到整机

从电、二极管、逻辑门和时钟到最小 CPU 与整机硬件

CPU Books

本书沿着器件、电平、逻辑、状态、时钟、算术、数据通路、控制器、最小 CPU、主存、总线和 I/O 的顺序，把一台计算机从最小可理解部件推导出来。

前言

这本书从硬件本身开始。全书首先建立底层合同：一根线上连续变化的电压怎样被稳定解释成 0 和 1；一个器件怎样让电流更容易沿某个方向通过；一个受控开关怎样让一个信号决定另一条电流路径是否导通。只有这些合同成立，后续的逻辑门、寄存器、ALU、CPU 和整机接口才具有可靠含义。

本书按照四个较厚章节展开。第一章把电平、器件、开关、逻辑门和组合电路放在同一条链路中讨论，说明 bit 如何从物理电平进入可计算的逻辑网络。第二章集中讨论状态、时钟与同步边界，说明硬件如何保存过去，并在清楚的时钟边界提交下一值。

第三章讨论数值解释、算术电路、ALU、数据通路和控制器。它把固定宽度 bit、补码、标志位、选择器、寄存器阵列和控制 trace 统一起来，使读者能够从旧状态走读到新状态。第四章再把这些部件合成为 CPU，并继续连接主存、cache、总线、I/O、DMA、中断和启动硬件，同时借助 8086、80386 等经典芯片观察真实处理器的历史剖面。

这种组织方式刻意避免把 CPU 作为孤立名词提前给出。CPU 在本书中是前述硬件部件按同步边界组织后的结果：PC 保存下一条编码地址，译码逻辑产生控制线，数据通路执行计算和写回，外部接口把处理器连接到主存与设备。读完整本书后，读者应能把一台机器从电平、开关、门电路、组合逻辑、时序逻辑、运算部件、数据通路、控制器一路连到整机。

本书采用接近 CSAPP (Computer Systems: A Programmer's Perspective) 等经典系统教材的标注方式：每讲一个抽象，都同时给出机器层证据。读逻辑门时同时检查真值表和开关路径；读组合逻辑时同时检查函数、延迟和毛刺；读寄存器时同时检查 bit 和采样边界；读 CPU 时同时检查指令、控制信号和数据通路 trace。章节末尾的“经典教材标注”不是附加材料，而是用于检验硬件理解是否已经落到证据层面。

目录

第一部分 从电平到数据通路	1
第 1 章 电平、开关、逻辑门与组合电路	3
一、从一根线得到可靠 bit	4
二、从器件到受控电流路径	9
三、从路径表得到逻辑门	13
四、门网络构成组合逻辑	18
五、延迟、负载、毛刺与验收	25
第 2 章 状态、时钟与同步边界	32
一、反馈、锁存器与保存一个 bit	32
二、触发器、时钟周期与采样窗口	32
三、关键路径、复位与跨边界输入	33
四、寄存器、计数器与移位结构	34
五、状态机把硬件动作切成阶段	34
第 3 章 数值、ALU、数据通路与控制	36
一、固定宽度 bit 的数值解释	36
二、ALU 把候选计算收进一个组合核心	37
三、数据通路让值从旧状态走到新状态	38
四、控制器输出一组一致的控制线	39
五、用控制 trace 走读一次动作	40
第二部分 最小 CPU 与整机硬件	42
第 4 章 CPU、经典芯片与整机硬件	44
一、从动作编码到最小 CPU	44
二、8086：从芯片接口看早期微处理器	45
三、80386：32-bit、保护模式与地址转换	46
四、主存、总线、I/O 与 DMA	46
五、整机启动与经典芯片后的演进	47

从电平到数据通路

本部分导读

本部分集中回答一个连续问题：怎样从可控电流路径得到稳定的 0 和 1，再把多个 0/1 组合成可计算的逻辑网络；怎样让硬件保存状态、按时钟边界推进；怎样把数值解释、ALU、寄存器阵列、数据通路和控制信号组织成可重复的硬件动作。

第 1 章

电平、开关、逻辑门与组合电路

本章将“电和器件”“开关到门”“组合逻辑”合并为一条完整链路：连续变化的电压先被解释成稳定的 0/1；二极管和受控开关提供可预测的电流路径；开关网络形成 NOT、NAND、NOR、XOR 等门；门再组合成判断器、加法器、选择器和译码器。这样的组织方式有助于读者在同一章内建立从物理电平到组合电路的连续推导。

本章不展开完整的半导体物理，也不把内容处理成布尔代数速查表。它关注一个核心问题：一根真实导线上的电压，怎样逐步成为 CPU 可使用的 bit 计算。后续所有寄存器、ALU、数据通路、控制器、内存和 I/O，最终都必须满足这里建立的底层合同。

为避免把内容讲成分散概念，本章使用一个贯穿案例：一块简化的安全启动控制板。它接收启动按钮、保护盖、急停、温度报警和电流报警，输出启动允许、电机使能和故障灯。这个小系统不涉及软件，也不需要处理器；它只需要把输入电平可靠地解释成 bit，再用门网络形成输出。正因为它足够小，读者可以逐项检查每根线的默认状态、每个门的路径、每个输出的电平和每条组合路径的延迟。

信号	含义	需要检查的硬件问题
start_button	人按下启动按钮	松开时是否悬空，按下时是否可靠为 1
cover_closed	保护盖已经闭合	断线或接触不良时应落到安全默认值
emergency_ok	急停没有被按下	信号名为正逻辑，但实物开关可能采用低有效接法
temp_alarm	温度报警	任一报警都应阻止启动并点亮故障灯
current_alarm	电流报警	报警输入不能依赖偶然电平或悬空节点
allow_start	允许启动	必须由所有安全条件共同决定
motor_enable_cmd	电机使能命令	只能由已经验收过的组合结果产生
fault_lamp	故障灯	需要汇总多个故障证据

核心理念

数字硬件的第一层抽象不是门，而是带余量的电平；数字硬件的第一层实现不是公式，而是受控电流路径。

本章还提前引入几个芯片参照。它们暂时不作为完整处理器来讲，而是作为本章概念的落点：门电路怎样被封装成标准逻辑芯片，组合网络怎样扩展成处理器内部的数据通路，现代芯片又怎样把同样的逻辑函数做得更快、更密、更低时延。

芯片或芯片族	可观察对象	本章先借用的概念
7400 系列 TTL	标准 NAND、NOR、触发器和计数器	门不是符号，而是有电平阈值、扇出和延迟的器件
4000 系列 CMOS	低静态功耗的 CMOS 逻辑门	互补上拉/下拉路径与稳定状态功耗
Intel 8086	早期 16-bit x86 微处理器	门、PLA、微码和总线控制组合成完整 CPU

Intel 80386	32-bit x86 处理器	更多晶体管带来更宽数据通路、更复杂控制和更高频率
现代 CPU/SoC	多级缓存、流水线、片上互连和电源管理	低时延来自器件、布局、体系结构和存储层次共同作用

例如，7400 系列里的 NAND 门可以直接对应本章的 NAND 真值表，但真正芯片还会给出**输入阈值、输出电流、传播延迟和扇出限制**；8086 可以看成许多门、寄存器、选择器、译码器和控制逻辑组成的早期微处理器；80386 则把 x86 推进到 32-bit 数据通路和更复杂的系统控制。现代 CPU 仍然由同类基本构件组成，只是晶体管更小、布局更紧、缓存更近、流水线和预测机制更复杂。把这些案例提前放在这里，是为了避免读者把第一章误解成“只讲离散门电路”；本章讲的是所有芯片共同依赖的底层合同。

一、从一根线得到可靠 bit

电路里说“一根线是 1”，完整含义不是“线上写着数字 1”，而是“这个节点相对于参考地的电压落在接收端承认的高电平区间内”。节点可以理解成多条金属连接汇合成的同一个电气位置；参考地是整块电路共同约定的 0V 参考点；电压是两个点之间的差值。数字电路里说某根信号线是 0V、3.3V 或 5V，实际都是在说它相对于地的电压。

电流说明电荷正在沿通路流动。若输出节点被一条通路接向电源，它可能被拉到高电平；若被一条通路接向地，它可能被拉到低电平。若两条通路都断开，节点不会自动变成 0，而是可能暂时停在旧电压附近，再被漏电流、干扰、后级输入或测量探针慢慢推走。若上拉和下拉两条强通路同时打开，电源到地之间形成冲突电流，电平不可靠，还会增加功耗和发热。

需要先区分导线和程序变量。变量没有被赋值时，语言可以规定默认值；导线没有被驱动时，只剩物理过程。它可能因为寄生电容保存旧电压，表现为保留上一次状态；也可能被旁边切换的信号线、人体接近、探针输入阻抗或后级漏电流轻微影响。调试数字硬件时，不能把这种偶然稳定误认为设计正确。

以按钮输入为例。假设按钮按下表示“请求发生”。一种不完整的接法是按钮一端接信号线，另一端接电源：

```
button pressed -> signal connected to VCC
button released -> signal connected to nothing
```

按下时，信号线被电源路径拉高，该部分行为符合预期。松开时，信号线并没有被明确拉低，它只是断开了。这个节点可能保留先前电荷，也可能被旁边导线耦合出一定电压，还可能被后级输入漏电流拖向某个不确定位置。于是接收端可能读到 1、0 或过渡电压。问题的根源不是程序错误，而是电路没有为节点规定明确归宿。

正确的第一层想法是：任何要被后级判断的输出节点，都必须在每一种输入情况下被明确地拉向高电平或低电平。按钮松开时需要一条弱下拉路径把信号线拉向地；按钮按下时，按钮路径把它拉向电源。弱下拉不是为了在按钮按下时和按钮争夺电源，而是为了在按钮松开时给节点一个默认归宿。它太弱，节点下降慢、抗干扰差；它太强，按钮按下时浪费电流，甚至让高电平不够高。

场景	高电平路径	低电平路径	结论
按下	按钮把节点接向电源	弱下拉仍存在	可靠高
松开	高电平路径断开	弱下拉把节点接向地	可靠低
接触抖动	高电平路径快速断续	弱下拉持续存在	需要消抖
导线断裂	按钮路径永远断开	若下拉还在，则为低	有默认值
没有下拉	高电平路径断开	低电平路径也断开	悬空

该表给出硬件读法的雏形。它不只问逻辑上应该是 0 还是 1，还问路径在哪里、

谁驱动谁、异常时会落到哪个默认状态。后面讲复位、总线仲裁和设备中断时，同样要用这种检查法：每个状态都要有明确来源，每个默认值都要有物理路径支持。

把按钮放回安全启动控制板中，可以得到更明确的安全要求。启动按钮松开时，系统不应因为线缆悬空而误认为有人按下；保护盖传感器断线时，系统不应默认保护盖闭合；报警线断开时，系统不应静默地继续允许运行。因此，“默认值”不是随意选择，而是安全策略的一部分。对人机输入而言，常见策略是让未动作、断线或未知状态落到不启动、不使能或报警一侧；对内部数据通路而言，策略可能不同，但仍必须明确。

输入线	不完整设计的风险	更稳妥的默认方向
启动按钮	松开时悬空，可能误触发	默认不启动，按下才变为启动请求
保护盖检测	断线时可能被误读为闭合	默认未闭合，需要传感器明确证明闭合
急停链路	接触不良时可能丢失急停证据	默认不允许运行，需要链路明确证明急停正常
报警输入	断线时可能丢失报警	根据安全需求选择默认报警或要求上层诊断

电平也不能理解成两个数学点。假设一块小电路用 5V 供电。线上的真实电压可能是 0V、0.17V、1.3V、2.6V、4.8V，也可能在切换过程中从 0V 慢慢升到 5V。硬件不能要求每一级都精确读出这个连续数值。更可靠的做法是规定两个稳定区间：低到足够接近地电位时解释为 0，高到足够接近电源电位时解释为 1，中间一段不作为长期稳定状态。

电压范围	逻辑解释	设计意义
接近地电位	逻辑 0	小噪声不会轻易把它推成 1
中间区域	不作为稳定状态	切换可以经过，但不能长期停留
接近电源电位	逻辑 1	小噪声不会轻易把它拉成 0

可以用四个电压指标描述两级电路之间的合同。 V_{OH} 是输出端保证的高电平下限， V_{OL} 是输出端保证的低电平上限； V_{IH} 是输入端承认为高的最低电压， V_{IL} 是输入端承认为低的最高电压。若一个电路保证 V_{OH} 高于下一级需要的 V_{IH} ，中间差额就是高电平噪声余量；若 V_{OL} 低于下一级允许的 V_{IL} ，中间差额就是低电平噪声余量。

符号	含义	读法
V_{OH}	输出保证为高时的最差电压	前一级说：我给出的 1 至少高到这里
V_{IH}	输入承认为高的最低电压	后一级说：至少给到这里，我才认作 1
V_{OL}	输出保证为低时的最差电压	前一级说：我给出的 0 至多高到这里
V_{IL}	输入承认为低的最高电压	后一级说：低到这里以下，我才认作 0

例如，某一级输出保证高电平至少为 2.4V，后一级输入只要求达到 2.0V 就承认为高，那么高电平噪声余量为 0.4V。若同一输出在最坏负载、温度和工艺条件下只能到 1.9V，逻辑表达式仍然可能写着“输出为 1”，但后一级不会可靠承认这个 1。低电平也同理：前级输出低电平的最坏值必须低于后一级允许的低电平上限，并留出余量。

项目	前级保证	后级要求	结论
----	------	------	----

高电平	$V_{OH} \geq 2.4V$	$V_{IH} \geq 2.0V$	余量 0.4V
低电平	$V_{OL} \leq 0.4V$	$V_{IL} \leq 0.8V$	余量 0.4V
失败情形	实测高电平仅后级需要 2.0V		不能可靠读作 1
	1.9V		

数字抽象并不是忽略模拟世界，而是把模拟世界约束在合同之内。只要所有连接都满足这些最差条件，后续讨论 bit、真值表和指令编码才有可靠基础。若这些条件不满足，系统层面可能出现间歇性现象：同一块板有时能启动、有时不能；按键有时触发两次；某条总线在高温下出错；更换更长排线后原本稳定的电路开始误判。这些现象不一定来自逻辑表达式错误，也可能来自电平合同被破坏。

输出还必须能驱动下一级。一个节点如果只能在空载时看起来像 1，一接上后级就被拖低，它不是可靠数字输出。后级输入、板级走线、封装和探针都有寄生电容。把一个节点从 0 拉到 1，本质上是在给这个电容充电；把它从 1 拉到 0，本质上是在放电。驱动路径越弱，等效电阻越大；负载越多，等效电容越大；上升和下降越慢。

```
larger load capacitance -> slower edge
weaker driver           -> slower edge
slower edge             -> less time margin before sampling
```

所以示波器上看到的真实信号不是理想直角。电压边沿有斜率，可能有过冲、振铃、平台和噪声。逻辑 0/1 的背后是一条随时间变化的电压曲线，接收端只是在某个时刻把它归类为 0 或 1。若这个时刻到来之前电压还没有进入可靠区间，最终会稳定也不够。

因此，一个 bit 被称为可靠，至少需要同时满足四项条件。第一，有明确驱动路径；第二，电平进入接收端承认的稳定区间并留有噪声余量；第三，驱动端能够带动实际负载；第四，在后级采样或使用之前，信号已经稳定足够长时间。

验收项	要求	失败表现
驱动路径	每种输入情况下都能拉向高或低	悬空、保留旧值、被干扰改变
电平余量	最坏输出满足后级输入阈值	高低电平被误判或边界不稳定
驱动能力	输出能带动后级输入和走线电容	边沿变慢、电压被拖低
稳定时间	后级使用前已经越过阈值并稳定	最终值正确但采样时刻错误

外部输入还需要调理。按钮和机械触点会抖动，线缆会引入噪声，传感器输出可能是开漏或开集结构，板外信号还可能与板内电源域不同。第一章暂不展开完整接口电路，但必须建立原则：进入组合逻辑核心之前，外部输入应先变成带默认值、带驱动能力、带明确有效电平的内部信号。否则后面的真值表虽然正确，输入源本身却可能不可靠。

外部现象	进入组合逻辑前的处理	不处理的后果
按钮抖动	消抖或在后续同步边界过滤	一次按下可能产生多次请求
长线噪声	上拉/下拉、滤波、缓冲或屏蔽	误触发、边沿变慢、阈值附近摆动
开漏输出	外部上拉并限制总线负载	释放时没有可靠高电平
电源域不同	电平转换或隔离	后级阈值不匹配甚至损坏器件

因此，书写硬件规格时应区分“原始外部信号”和“内部逻辑信号”。原始外部信号属于板外世界，可能带有抖动、噪声、线缆压降和接插件故障；内部逻辑信号属于组合逻辑世界，必须已经具有明确的 0/1 含义。若把二者混成一个名字，读者会误

以为真值表直接约束了外部物理现象。

信号层次	典型名称	设计要求
外部触点	<code>start_button_raw</code>	允许抖动,但必须有默认路径和保护边界
输入调理后	<code>start_button_clean</code>	在逻辑核心看来只能是稳定 0 或稳定 1
语义信号	<code>start_request</code>	明确表示“本周期有启动请求”这一逻辑事实
安全汇总	<code>allow_start</code>	由全部安全条件共同决定,不能依赖原始触点

施密特触发输入是常见的边界处理方式。普通输入通常只有一个判断阈值附近的过渡区;当信号边沿很慢或带噪声时,电压可能在阈值附近反复穿越,导致接收端多次翻转。施密特触发器使用两个不同阈值:上升时必须超过较高阈值才承认为 1,下降时必须低于较低阈值才承认为 0。两个阈值之间的间隔称为滞回。滞回不是改变逻辑功能,而是让输入在噪声环境中更稳定地跨过边界。

输入处理	适用情形	对可靠 bit 的贡献
上拉或下拉 限流和保护	按钮、开漏输出、断线默认值 板外连接、误插、瞬态干扰	给悬空节点明确归宿 避免异常电压直接进入逻辑核心
滤波或消抖	机械触点、低速传感器	把多次物理跳变收敛为一次逻辑事件
施密特触发	慢边沿、噪声叠加的输入	用滞回减少阈值附近的反复翻转
电平转换	1.8V、3.3V、5V 等混合系统	让输出合同匹配输入合同

按钮消抖尤其适合作为入门反例。人只按下一次,触点却可能在几毫秒内多次接通、断开。如果后级把每一次跳变都看成一次请求,一个启动按钮就可能产生多个启动脉冲。消抖电路或后续时序逻辑的职责,是在确认输入持续稳定后才更新内部请求信号。

时间段	原始触点	内部请求	解释
按下前	稳定断开	0	默认下拉给出明确低电平
刚按下	断续跳变	保持 0	尚未接受为一次稳定请求
稳定按下	持续接通	1	已满足消抖条件
刚松开	断续跳变	保持 1 或等待释放 确认	不把抖动解释为多次按键
稳定松开	持续断开	0	回到默认状态

这组处理说明一个原则:逻辑核心应当面对已经被整理过的事实,而不是直接面对物理噪声。安全启动控制板中,`allow_start` 不应直接依赖 `start_button_raw`,而应依赖 `start_request`;故障汇总不应直接依赖线缆上的未定义电平,而应依赖经过默认值、阈值和驱动能力检查后的报警信号。后面进入触发器和时钟边界后,还会继续把这种“边界整理”推广到同步采样。

可靠 bit 还必须考虑上电和复位。电源从 0V 上升到额定电压需要时间,芯片内部阈值、外部上拉下拉、时钟源和输入调理电路不一定同时进入正常状态。在这段时间里,若某个输出直接控制电机、继电器或写使能,就不能假设它已经是安全值。正式硬件设计通常要求复位链路或电源监控电路在电源稳定之前保持关键输出

禁止。

阶段	硬件状态	对关键输出的要求
断电	节点可能通过漏电或外部连接获得微弱电压	不应驱动执行机构
电源爬升	阈值、时钟和输入调理尚未完全有效	复位或禁止链路保持有效
复位保持	内部状态被强制到约定默认值	<code>motor_enable_cmd=0</code> ，写使能关闭
复位释放	输入和时钟达到有效条件	组合逻辑开始按规格产生输出
正常运行	所有电平合同和时序合同成立	后级只使用已经稳定的信号

这也是“默认安全”的电气含义。它不是在文档里写一句“默认不启动”，而是要能指出：电源未稳时谁拉住使能，复位未释放时谁关闭输出，输入断线时谁把条件解释为不满足，组合路径未稳定时谁阻止后级采样。若这些问题回答不出来，系统可能在最危险的上电瞬间出现短促误动作。

最后，可靠 bit 应该能被测量证据支持。万用表可以检查静态电压，却看不见窄脉冲和抖动；示波器能观察边沿、过冲、振铃和噪声；逻辑分析仪适合记录长时间数字事件，但它同样依据自己的阈值把模拟电压解释成 0/1。测量工具不是抽象之外的附属品，而是把电平合同落到实物上的证据来源。

工具或证据	能说明什么	不能单独证明什么
原理图审查	是否有默认路径、保护和驱动边界	实物焊接和噪声环境是否合格
万用表	静态高低电平是否大致正确	边沿速度、毛刺和短时抖动
示波器	波形、阈值穿越、过冲和振铃	长时间低概率事件一定不会发生
逻辑分析仪	数字事件顺序和持续时间	模拟电平余量是否足够
数据手册参数	最坏阈值、延迟、负载能力	板级实现是否满足这些边界

同一根按钮输入线，可以用三类工具得到互补证据。假设 `start_button_raw` 采用上拉输入，按下时接地，内部逻辑再把它反相为 `start_request`。万用表能确认松开和按下时的静态电压方向；示波器能观察按下瞬间的触点抖动、边沿速度和阈值穿越；逻辑分析仪能记录内部请求是否只产生一次事件。三者关注的问题不同，不能互相替代。

证据来源	在按钮案例中应看到什么	若证据异常应怀疑什么
万用表	松开为稳定高电平，按下为稳定低电平	上拉/下拉错误、接线反向、触点失效
示波器	按下和松开有有限抖动，最终越过阈值并稳定	抖动过长、边沿太慢、噪声跨越阈值
逻辑分析仪	消抖后只出现一次 <code>start_request</code>	消抖不足、阈值设置不当、采样边界错误
数据手册	输入阈值、滞回、电流和最大额定值满足设计	电平合同只在实验样品上偶然成立

这类证据写入设计文档后，第一章的抽象就不再停留在“按钮产生 0/1”。更完整的说法是：外部触点通过默认路径得到静态归宿，通过输入调理穿过阈值，通过消抖变成一次语义事件，并在后级使用前满足电平和时间条件。后面的内存读写、总线请求和中断输入，也应按同样方式建立边界证据。

二、从器件到受控电流路径

二极管可以先理解成方向性很强的器件。在合适偏置下，它比较容易让电流沿一个方向通过；反向偏置时，它基本阻止电流通过。这个模型不需要一开始展开半导体物理，却已经足够说明硬件设计的一条基本路线：利用器件物理特性，制造可预测的电流路径。

二极管的“单向”不是理想数学开关。正向导通通常需要一定压差，导通后还有压降；反向并非绝对没有电流，而是漏电很小；电压过高时还可能击穿。初读数字硬件时可以先用理想模型建立方向感，但不能忘记真实器件会吃掉电压余量、引入延迟和功耗。

二极管事实	对数字电路的影响	读书时要追问
有方向性	可以限制电流路径	这条路径在何种输入下导通
有正向压降	输出电平会损失一部分余量	级联后高电平还够不够高
有漏电和极限	不能把反向阻断看成无限好	极端温度、电压下是否仍可靠
不能主动放大	不能恢复变弱的电平	下一级是否需要缓冲或开关恢复

二极管逻辑可以表达某些简单关系。若有两路输入想共同点亮一个指示节点，任意一路为高时都能通过二极管把输出节点拉高，输出就呈现 OR 行为：

```
if A can drive through a diode:
    Y may be pulled high
if B can drive through a diode:
    Y may be pulled high
```

该例能够说明方向性逻辑，但也暴露出局限。第一，二极管会带来压降，输出高电平可能比输入高电平低一截；级联多了，电平余量会越来越紧。第二，二极管只能提供方向性，不能主动把一个偏弱的高电平重新恢复成强高电平。第三，它不方便实现取反，而取反是构造通用逻辑必须具备的能力。

以安全启动控制板的故障汇总为例，`temp_alarm` 和 `current_alarm` 都可能点亮 `fault_lamp`。二极管 OR 可以让任一报警输入把故障节点拉高，但如果这个故障节点还要继续驱动后面的禁止逻辑、记录逻辑和指示灯，问题就会扩大。每经过一级二极管，电平余量都可能被吃掉一部分；每增加一个负载，边沿都会变慢。最终出现的不是“公式错了”，而是高电平不够高、上升不够快、后级读不稳。

二极管 OR 用法	可以解决的问题	不能解决的问题
汇总多个报警源	任一报警能提供一条拉高路径	不能主动恢复被削弱的高电平
隔离多个输入源	一个输入不容易反向灌入另一个输入	正向压降会消耗电平余量
形成简单有线关系	结构直观，适合早期理解	不适合无限级联成通用逻辑网络

取反为什么重要？因为许多条件天然需要反向解释。传感器未触发时允许运行，触发时禁止运行；按钮未按下时保持，按下时改变。若电路只能表达“有一路导通”，却不能把 1 变成 0、把 0 变成 1，那么可组合的逻辑很快就受限。要继续向前，需要一个信号能控制另一条电流路径是否导通。

晶体管的细节很深，但作为第一层硬件模型，可以先把它当成受控开关。控制端不是直接给输出供电，而是决定输出节点到某条电源路径之间是否接通。受控开关和普通机械开关最大的差别，是控制信号本身也是电信号。按钮需要人按，晶体管可以由前一级逻辑输出控制。于是硬件有了“信号控制信号”的递归能力：一个门的输出可以控制下一批开关，下一批开关再形成新的输出。

在 CMOS 数字电路中，可以先用极简模型区分两类受控开关。NMOS 更适合把节

点拉向地，输入控制为高时容易导通；PMOS 更适合把节点拉向电源，输入控制为低时容易导通。真实器件还有阈值电压、体效应、漏电、沟道电阻和寄生电容等细节，但第一章暂时只抓住一件事：上拉路径和下拉路径由互补器件协作，才能把输出恢复为强 0 或强 1。

器件视角	在门电路中的典型职责	需要避免的误解
NMOS 下拉	给输出提供到地的受控路径	不是“产生 0”这个符号，而是放电路径
PMOS 上拉	给输出提供到电源的受控路径	不是“产生 1”这个符号，而是充电路径
互补配对	一个负责拉低，另一个负责拉高	不能让两条强路径长期同时导通
输入过渡	两类器件可能都部分导通	过渡区只应短暂经过，不应长期停留

反相器可以作为第一种完整逻辑门来理解。输入低时，上拉路径导通、下拉路径断开，输出被拉高；输入高时，上拉路径断开、下拉路径导通，输出被拉低。注意这里不是“输入低所以输出自动高”，而是“输入低使某个高电平路径导通”。把每一句逻辑描述翻译成路径描述，能避免很多硬件误解。

输入	上拉路径	下拉路径	输出
低电平	导通	断开	被拉高
高电平	断开	导通	被拉低
过渡中	可能部分导通	可能部分导通	暂时不应采样

第三行很重要。真实输入从低到高不是瞬间跳变，在过渡区间里，上拉和下拉可能都不处于理想开/关状态。短时间内出现电源到地的电流并不奇怪，这就是动态功耗和短路功耗的一部分。只要过渡足够快、采样避开中间区域，数字抽象仍然成立。若输入边沿太慢，电路可能在中间区域停留过久，功耗和误判风险都会上升。

把开关串联，可以表达“两个条件都满足才有通路”；把开关并联，可以表达“任意一个条件满足就有通路”。以下拉网络为例，两个受控开关串联，只有 A 和 B 都让开关导通时，输出节点才会被拉向低电平；两个开关并联，只要 A 或 B 有一个导通，输出节点就会被拉低。上拉网络通常要和下拉网络互补，保证输出在稳定输入下要么被拉高，要么被拉低，而不是两边都断或两边都强通。这种“互补路径”思想，后面会自然走到 CMOS 逻辑。

互补路径可以用两条规则记住。第一，输出应该被明确驱动：下拉网络不导通时，上拉网络应当负责把输出拉高；下拉网络导通时，上拉网络应当关闭，避免电源到地的强冲突。第二，稳定输入下不应依赖中间电压：如果上拉和下拉都处于部分导通状态，输出可能落在不可靠区间，功耗也会上升。CMOS 逻辑之所以适合作为数字门的基础，关键就在于它把“拉高”和“拉低”组织成互补关系，并在稳定状态下尽量避免静态直通电流。

路径状态	输出后果	设计判断
上拉开、下拉关	输出被拉向高电平	合法稳定状态
上拉关、下拉开	输出被拉向低电平	合法稳定状态
上拉关、下拉关	输出悬空，可能保留旧电压	需要补默认路径或改逻辑
上拉开、下拉开	电源到地形成强通路	功耗高，电平不可靠
两边部分导通	输出可能处于过渡区	只允许短时间经过，不应长期停留

从功耗角度看，数字门并非“只有 0 和 1，没有模拟代价”。稳定状态下，如果

只有上拉或只有下拉导通，静态电流可以很小；切换时，输出节点的寄生电容需要充电或放电，形成动态功耗；输入经过过渡区时，上拉和下拉可能短时间同时导通，形成短路功耗。后面讨论频率、负载和关键路径时，这些现象会继续出现。

这也是 CMOS 成为主流数字逻辑基础的重要原因。它不是没有功耗，而是在稳定 0 和稳定 1 时尽量避免持续直通电流，把主要能量消耗集中到节点切换和短暂过渡中。随着芯片集成度提高，设计者更关心哪些节点频繁翻转、哪些长线负载过大、哪些门需要更强驱动，以及哪些路径应由寄存器边界切分。由此可见，第一章的受控路径模型会直接延伸到后面的频率、功耗和时序预算。

功耗来源	物理原因	设计含义
静态功耗	漏电或稳定状态下仍存在电流路径	器件越小越不能忽略漏电和待机功耗
动态功耗	输出节点和连线电容反复充放电	高翻转率、长线和大扇出都会增加能耗
短路功耗	输入过渡时上拉和下拉短暂同时导通	慢边沿不仅影响时序，也会增加功耗
驱动损耗	输出级推动外部负载或长线	指示灯、总线和连接器常需要专用驱动级

开漏或开集结构是理解共享线路的重要例子。一个输出器件只负责把线拉低，释放时并不主动拉高；高电平由公共上拉电阻提供。多个器件可以共享同一根线，只要任意一个器件拉低，整根线就是低电平；只有所有器件都释放，线上才被上拉为高电平。这种结构常用于中断请求、故障汇总或简单总线。

共享线状态	物理路径	逻辑后果
所有输出释放	上拉电阻把线拉高	线为 1，但上升速度取决于上拉和负载
任一输出拉低	该输出提供到地路径	线为 0，多个输出不会互相强冲突
上拉过弱	高电平上升慢	采样前可能尚未达到高阈值
上拉过强	拉低时电流大	功耗增加，输出器件负担加重

这说明“多个来源接到一根线”并非永远错误，关键在于是否有协议和电气结构保证同一时刻不会出现强驱动冲突。开漏共享线允许多个设备共同拉低，是因为它们只竞争“是否提供到地路径”，不竞争“谁主动拉高”。普通推挽输出若直接并在一起，一个输出拉高、另一个输出拉低，就会形成强冲突。

三态输出是另一类共享线路方案。普通数字输出只有主动拉高和主动拉低两种驱动状态；三态输出还可以进入高阻态。高阻态不是逻辑 0，也不是逻辑 1，而是输出驱动器基本断开，让这一路暂时不参与驱动总线。若多块设备共享数据总线，控制逻辑必须保证同一时刻最多只有一个设备打开输出驱动，其余设备都处于高阻态。否则，一个设备驱动 1、另一个设备驱动 0，仍会发生总线争用。

输出方式	驱动状态	适合表达的问题
推挽输出	主动拉高或主动拉低	单一来源驱动一条内部信号
开漏输出	只主动拉低，释放靠上拉	多个来源共同提出低有效请求
三态输出	拉高、拉低或高阻	多个来源按仲裁结果轮流占用总线

开漏和三态都要求协议。开漏协议通常是“任一设备可以拉低，所有设备释放后才为高”；三态协议通常是“仲裁者只允许一个设备驱动，其他设备必须释放”。这两类结构后来会在中断线、片选、数据总线和 I/O 控制中反复出现。现在先记住硬件

判断标准：多来源共享一根线时，必须说明每个来源什么时候能驱动、什么时候必须释放，以及释放后由谁给出默认电平。

把这些器件观点收束起来，可以得到本章的第二层抽象：数字门不是画在纸上的符号，而是一组受输入控制的上拉、下拉、释放和恢复路径。二极管提供方向性，但不恢复电平；晶体管提供受控路径；互补开关网络提供明确的高低驱动；缓冲和三态驱动器提供负载隔离和共享边界。后面所有更大的组合电路，仍然要回到这些路径检查。

器件族之间还存在电平兼容问题。即使两个器件都叫“数字逻辑”，也不表示它们可以任意直连。一个 5V 供电器件的输出、一个 3.3V CMOS 输入、一个 TTL 兼容输入和一个开漏传感器，可能有不同的高低阈值、输入电流和输出驱动能力。设计者必须把数据手册中的最坏输出和最坏输入放在同一张表里比较，而不是只看“都是 0/1”。

连接检查	要问的问题	典型风险
电源电压	前级输出高电平是否超过后级输入上限	过压损坏或长期可靠性下降
输入阈值	前级最差高/低输出是否满足后级阈值	电平看似变化，后级不承认
输入电流	前级能否提供或吸收后级所需电流	电平被拖偏，余量不足
输出驱动	前级能否驱动全部负载和走线电容	边沿变慢，时序失败
上拉配置	开漏/开集输出是否有合适上拉	释放后没有可靠高电平

数据手册里的参数要按“最坏组合”阅读。若前级在轻载、常温下能输出漂亮的 3.3V，不代表它在最大负载、高温、低电源电压下仍有同样余量；若后级典型阈值较低，也不代表所有批次、所有温度下都按典型值判断。经典教材强调抽象层次，但并不鼓励忽略边界条件。抽象的意义是把边界写清楚，然后在边界内使用更高层概念。

从这个角度看，缓冲器、电平转换器和驱动器并不是“逻辑上多余”的器件。缓冲器可以恢复电平和增强驱动；电平转换器让两个电源域的合同重新匹配；驱动器让小逻辑信号控制更大的负载。它们是否改变真值表不是唯一问题；它们是否让真实节点满足电平、负载和时序合同，才是硬件设计要回答的问题。

标准逻辑芯片把这些约束以产品形式暴露出来。以常见的 7400 系列为例，一个封装中可以放入若干个 NAND 门。教材里的 NAND 真值表只说明稳定逻辑行为，而芯片数据手册还会说明每个输入能承认的高低电平、输出能提供或吸收的电流、传播延迟范围、工作电压范围和扇出建议。换言之，标准门芯片把“布尔函数”和“电气合同”绑定在一起出售。

从 7400 类门芯片读到的项目	对应本章概念	设计时的含义
输入高低阈值	可靠 bit	前级输出必须落入后级承认区间
输出电流能力	驱动能力和扇出	一个门不能无限驱动后级
传播延迟	组合路径时序	多级门相连后要累计最坏延迟
工作电压范围	电平合同边界	不同电源域不能随意直连
封装引脚	物理边界	逻辑门最终仍要通过真实引脚连接

4000 系列 CMOS 逻辑则更适合说明互补路径和功耗。CMOS 门在稳定 0 或稳定 1 时，理想情况下没有从电源到地的持续强通路，静态功耗可以很低；切换时，输

出电容充放电，形成动态功耗。现代处理器虽然远比 4000 系列复杂，但仍然继承了这个核心事实：频率越高、切换节点越多、负载越大，动态功耗越高；门越小、线越短、负载越低，延迟越容易下降。

比较 7400 TTL 和 4000 CMOS 时，不应把它们只看成“都能实现 NAND/NOR”。TTL 器件常用于说明输入电流、扇出和标准逻辑电平；CMOS 器件更适合说明极高输入阻抗、低静态功耗和输入悬空风险。一个 CMOS 输入若没有明确上拉或下拉，可能因为极小漏电和耦合电荷保持在不确定电平；这与“输入阻抗高”并不矛盾，反而正是高阻输入必须定义默认值的原因。

芯片族视角	教材中应观察的事实	对本章的补充意义
7400 TTL	输入电流、输出灌/拉电流、门芯片是带电气合同的布尔函数	传播延迟和扇出限制
4000 CMOS	高输入阻抗、低静态功耗、宽电源范围和较慢边沿选型	默认路径和输入调理不能省略
高速 CMOS	更低延迟、更强驱动、更严格边沿和布局要求	速度提高后，负载、布线和毛刺更敏感

读一片标准 NAND 芯片，至少要同时读三层信息。第一层是逻辑：两个输入同时为 1 时输出为 0，其余情况输出为 1。第二层是电气：输入高低阈值、输出电流、工作电压和负载能力说明它能否与前后级可靠连接。第三层是时间：传播延迟说明输入变化后多久输出才可用。若只读第一层，就会把真实芯片误当成理想布尔符号；若只读第二、三层，又会失去门的功能目的。

三、从路径表得到逻辑门

门电路的第一步不是画符号，而是把输入组合列完整。两个输入 A、B 只有四种组合。电路必须对每一种组合给出明确输出，而且每一行都要满足前面的要求：输出节点不能悬空，不能上下冲突，必须能在规定时间内进入可靠电平区间。

从需求到门电路，建议固定采用五步。第一，给每个输入和输出写清楚硬件含义。第二，列出完整真值表，不跳过“看起来不会发生”的行。第三，标出输出为 1 的证据行，或者标出输出为 0 的禁止行。第四，把每一行翻译成开关路径：哪些条件使上拉导通，哪些条件使下拉导通。第五，检查每一行是否有明确驱动、是否存在冲突、是否满足电平和延迟约束。

步骤	目标	常见错误
定义信号	明确 0/1 分别表示什么	把低有效信号误读成高有效
列真值表	覆盖所有输入组合	只按自然语言想象少数情况
找证据行	明确输出为何为 1 或 0	漏掉边界输入或异常组合
写路径表	检查上拉、下拉和悬空	只写公式，不看电流路径
验收实现	检查驱动、延迟、功耗和毛刺	只证明逻辑等价，不证明硬件可靠

以安全允许灯为例。A 表示保护盖合上，B 表示急停未按下。允许灯亮的条件很严格：任一条件不满足，输出都必须为 0。

A	B	允许灯	为什么
0	0	0	盖子没合上，急停也不是可运行状态
0	1	0	急停条件满足，但盖子没合上
1	0	0	盖子合上了，但急停条件不满足

1	1	1	两个安全条件同时满足
---	---	---	------------

该表对应 AND 的行为。它不是抽象数学定义，而是“两个条件都满足才允许”的硬件约束。若某个实现让 $A=1$ 、 $B=0$ 时灯也亮，那就不是 AND 近似得不够好，而是安全行为错了。真值表的价值在于防止“只凭自然语言写电路”。“两个条件都满足”看起来简单，但实现时很容易漏掉某一行。

再看故障灯。A 表示温度报警，B 表示电流报警。故障灯的规则反过来：只要有一个坏消息出现，就要亮。

A	B	故障灯	为什么
0	0	0	没有任何故障证据
0	1	1	电流报警已经足够触发故障
1	0	1	温度报警已经足够触发故障
1	1	1	两个故障同时存在，仍应报警

该表对应 OR 的行为。AND 和 OR 的差别不在符号长什么样，而在需求不同：一个要求所有条件同时成立，一个要求任意证据成立。NOT 只有一个输入，但同样要列完整。若 A 表示“禁止信号”，输出 Y 表示“允许动作”，那么 $A=0$ 时 $Y=1$ ， $A=1$ 时 $Y=0$ 。它解决的是反向解释：输入出现时，输出反而关闭。

不要把“0 是假、1 是真”机械套到每个硬件信号上。某些信号是低有效，例如 `reset_n`、`chip_select_n`，低电平才表示动作发生。读这类信号时，先写真值表，再谈门电路。

低有效信号值得单独处理。名字中的 `_n` 常表示 active low，即低电平有效。若信号名为 `emergency_ok_n`，那么 0 可能表示“急停链路正常允许运行”，1 反而表示“不允许”。这种命名并不适合作为入门案例的默认读法，但真实硬件中很常见，因为开漏输出、有线汇总、复位链路和片选信号都可能倾向于低有效形式。读低有效信号时，不能先把 1 写成“发生”、0 写成“未发生”，必须先读信号合同。

信号名	有效电平	读法
<code>reset_n</code>	0	低电平时复位动作发生
<code>chip_select_n</code>	0	低电平时芯片被选中
<code>output_enable_n</code>	0	低电平时输出驱动器打开
<code>fault_n</code>	0	低电平可能表示故障请求有效，需看合同

低有效并不只是命名习惯，它会改变推理方向。假设外部急停链路使用低有效输入 `emergency_stop_n`：链路正常释放时为 1，急停动作、断线或安全继电器打开时为 0。若内部逻辑需要的是正逻辑 `emergency_ok`，就必须先做一次语义转换：

```
emergency_ok = emergency_stop_n
stop_request = NOT emergency_stop_n
```

这里 `emergency_ok` 和 `stop_request` 都可以是正确内部信号，但它们表达的事实不同。前者适合放进 `allow_start` 的 AND 链；后者适合放进故障或停机请求的 OR 链。若在同一个表达式里混用低有效原始信号和正逻辑语义信号，很容易写出方向相反的网网络。正式硬件文档通常会在信号表里明确有效电平，就是为了防止这种错误。

真值表说清楚“应该怎样”，路径表说明“为什么会这样”。用受控开关看 AND，最自然的结构是串联。两个开关串在同一条路径上，只有 A 和 B 都让自己的开关闭合时，整条路径才通。OR 更像并联。A 控制一条路径，B 控制另一条路径。只要任

意一路闭合，输出就能被拉向目标电平。

A	B	串联路径状态	AND 输出为什么成立或不成立
0	0	两个开关都断开	没有完整导通路径
0	1	A 断开，B 闭合	串联路径仍被 A 切断
1	0	A 闭合，B 断开	串联路径仍被 B 切断
1	1	两个开关都闭合	从源到输出的路径完整

如果只画一条“让输出为 1 的路径”，还没有完成门电路设计。输出节点不能靠悬空表示 0。因此每一行真值表还要补一张路径表，说明高电平路径和低电平路径的状态。一个合法门在稳定输入下，不应该让输出悬空，也不应该让上拉和下拉强路径同时导通。

从开关网络看，NAND 和 NOR 往往比 AND 和 OR 更像基础积木。原因是它们天然带反相输出，更容易形成互补的上拉和下拉结构：当下拉网络导通时输出被拉低；当下拉网络不导通时，上拉网络负责把输出拉高。

A	B	NAND 输出	开关直觉
0	0	1	下拉串联路径断开，上拉负责拉高
0	1	1	A 断开，下拉路径不完整
1	0	1	B 断开，下拉路径不完整
1	1	0	A、B 都闭合，下拉路径完整，输出被拉低

NOR 可以用同样方式读。NOR 的输出只有在 $A=0$ 且 $B=0$ 时为 1；只要 A 或 B 为 1，输出就为 0。若以下拉网络看，A、B 并联，只要任一路输入为 1，就能把输出拉低；只有两路都不导通时，上拉网络把输出拉高。

NAND 和 NOR 还可以帮助理解 De Morgan 定律。逻辑上， $\text{NOT } (A \text{ OR } B)$ 等价于 $(\text{NOT } A) \text{ AND } (\text{NOT } B)$ ；硬件上，这句话的含义是：只要 A 或 B 有一路故障证据成立，输出就应被拉低；只有 A 和 B 都没有故障证据，输出才允许被拉高。对于安全启动控制板，若 $\text{fault} = \text{temp_alarm OR current_alarm}$ ，那么“没有故障”就是 NOT fault ，也就是 $(\text{NOT temp_alarm}) \text{ AND } (\text{NOT current_alarm})$ 。这不是代数游戏，而是把“任一故障禁止启动”和“所有故障都不存在才允许启动”写成同一硬件约束的两种形式。

```
fault = temp_alarm OR current_alarm
no_fault = NOT fault
no_fault = (NOT temp_alarm) AND (NOT current_alarm)
```

只用 NAND 就能构造 NOT、AND、OR：

```
NOT A = A NAND A
A AND B = (A NAND B) NAND (A NAND B)
A OR B = (A NAND A) NAND (B NAND B)
```

这说明 NAND 在功能上完备，不说明所有实现都应该只用 NAND。真实设计会在功能正确的前提下，考虑门级数量、路径延迟、面积、功耗和负载。逻辑式等价，只说明稳定输出行为相同；它不保证延迟、功耗和驱动能力相同。

例如安全允许信号可以先写成：

```
allow_start = start_button AND cover_closed
              AND emergency_ok AND no_fault
```

若只用 NAND 实现，需要把多输入 AND 拆成若干 NAND 与反相结构。每拆一次，逻辑功能仍可保持，但门级层数、负载和延迟都会变化。因此“用 NAND 可以实现”

只是功能完备性结论；真正进入硬件设计时，还要继续检查最长路径和每一级扇出。

XOR 的输出在两个输入不同的时候为 1，相同的时候为 0。它解决的是“检测两个 bit 是否不同”这个具体问题。

A	B	A XOR B	解释
0	0	0	两个输入相同
0	1	1	两个输入不同
1	0	1	两个输入不同
1	1	0	两个输入相同

可以把 XOR 拆成两种会输出 1 的情况：A=1 且 B=0，或者 A=0 且 B=1：

$$A \text{ XOR } B = (A \text{ AND NOT } B) \text{ OR } (\text{NOT } A \text{ AND } B)$$

这条式子不要只当代数记忆。它对应两条证据路径：第一条证明“只有 A 为 1”，第二条证明“只有 B 为 1”。最后的 OR 合并这两条证据。若两个都为 0，没有证据成立；若两个都为 1，两条“只有一个为 1”的证据也都不成立。XOR 后面会反复出现：两个 bit 相加时，和位就是“两个输入是否不同”；比较两个 bit 时，XOR 可以告诉我们它们是否不同；接一个 NOT，就能得到“两个输入是否相同”。

XOR 也说明了“常用门”和“便宜门”不一定一致。在纸面表达式里，XOR 是一个符号；在门级实现里，它往往比 AND、OR、NOT 更复杂，路径延迟也可能更长。因此，读到加法器、比较器、校验位或地址译码时，不能只数逻辑符号数量。若关键路径里连续经过多个 XOR，速度代价可能比同样数量的简单门更高。经典教材在讲门延迟时强调这一点，是为了让读者把布尔函数和实现代价同时放进脑中。

真值表还可以用“最小项”来读。所谓最小项，就是一行输入组合对应的一条完整证据路径。例如三输入 A、B、C 中，只有 A=1、B=0、C=1 这一行输出为 1，则对应证据路径是 $A \text{ AND } (\text{NOT } B) \text{ AND } C$ 。把所有输出为 1 的行用 OR 合并，就得到一种直接实现。它不一定最简，但它有一个优点：每一项都能追溯到真值表中的具体一行。

A B C	输出 Y	对应证据路径
0 1 1	1	$(\text{NOT } A) \text{ AND } B \text{ AND } C$
1 0 1	1	$A \text{ AND } (\text{NOT } B) \text{ AND } C$
1 1 0	1	$A \text{ AND } B \text{ AND } (\text{NOT } C)$
1 1 1	1	$A \text{ AND } B \text{ AND } C$

若把这些最小项直接相加，就能实现“三个输入至少两个为 1”的判断。但前文给出的表达式 $(A \text{ AND } B) \text{ OR } (A \text{ AND } C) \text{ OR } (B \text{ AND } C)$ 更短，因为它把若干行合并成“任意两项同时成立”的证据。这里体现出两个层次：最小项保证从真值表机械得到正确表达式，化简则减少门数和路径深度。化简以后仍要确认没有改变低有效含义、默认安全策略和毛刺风险。

有时真值表里会出现“不关心”项。所谓不关心，不是设计者懒得定义，而是某些输入组合在系统合同中不会被使用，或者该组合出现时输出 0/1 都不会影响可观察行为。例如一个 2-bit 故障编码只允许 00、01、10，保留 11 作为非法状态；若后级在 11 时一定进入停机诊断，那么某个普通指示灯在 11 下亮或灭可能都可以接受。化简逻辑时可以利用不关心项减少门数，但必须谨慎：一旦所谓“不可能”的输入来自外部连接器、未初始化状态或故障状态，它就不应轻易被当作不关心。

输入组合类别	可以怎样处理	必须确认的问题
正常组合	严格定义输出	是否覆盖了所有业务行为
保留组合	可定义为诊断、停机或不关心	后级是否真的不会误用该输出
非法组合	通常导向安全默认值	硬件上是否可能短暂出现

外部故障组合 不应随意当作不关心 断线、噪声、上电瞬间如何表现

对于安全启动控制板，保守做法是：任何未确认安全的组合，都不应让 `allow_start` 为 1。即使某个组合在正常操作流程中不会出现，只要它可能由断线、上电初态、输入调理失败或编码错误产生，就应进入“不启动”或“故障”一侧。组合逻辑的化简必须服从系统合同，而不是反过来让系统合同迁就最短表达式。

逻辑化简还要保留可审查性。对教学和安全控制而言，表达式稍长但能清楚对应需求，往往比极度压缩的表达式更容易审查。若某个输出必须由四个安全条件共同允许，那么把表达式保留为四个语义项，有助于后续检查每个条件来自哪里、默认值是什么、是否经过输入调理。化简可以减少门数和延迟，但不能让规格意图变得不可见。

表达方式	优点	风险
逐项语义表达式 最小项展开	容易对应需求和安全审查 可机械追溯到真值表行	门数和层数可能不是最少 表达式可能很长，延迟不一定好
代数化简式	可能减少门数和路径层数	可能隐藏低有效、默认值和故障策略
器件库映射式	接近实际可实现结构	需要重新检查负载、扇入和时序

因此，正式设计常保留多个层次的描述：需求层写“所有安全条件满足才允许启动”；逻辑层写 `allow_start = start_request AND cover_closed AND emergency_ok AND no_fault`；实现层再说明由哪些门、缓冲和输出驱动器组成；验收层记录电平、延迟、毛刺和异常输入检查。四个层次互相对应，才能避免“公式正确但系统含义错误”。

处理器控制逻辑中常见的 PLA，可以提前理解为“可规则化实现的组合逻辑”。PLA 的基本思想是先用一组 AND 项识别输入条件，再用 OR 项合成输出控制信号。例如指令译码可以把操作码、状态位和时序阶段转成若干控制线：选择哪一路进入 ALU、是否写寄存器、是否读内存、下一步进入哪个控制状态。第一章不需要展开 PLA 版图，但读者应看出它和最小项方法的关系：先识别条件，再汇总证据。

PLA 视角	本章对应概念	在处理器中的用途
输入条件 AND 平面 OR 平面	真值表行、最小项、状态位 证据路径 证据汇总	操作码、标志位、时序阶段 某条指令或某类条件成立 产生写寄存器、读内存、选择 ALU 等控制线
输出控制	组合逻辑输出	驱动数据通路选择器和使能信号

8086 这类早期微处理器内部就包含大量译码和控制逻辑。它不是一堆孤立门的随意堆叠，而是把操作码解释、微操作顺序、总线周期和算术逻辑组合成可重复执行的机器动作。后面讲整机时会展开这些结构；在第一章，只需要先建立一个判断：处理器内部那些看似高级的控制信号，本质上也必须由可靠电平、门网络、组合路径和时序边界支撑。

8086 常被放在“完整 CPU”章节讲，但在第一章也能提前借用它的局部结构。它把取指相关的总线接口工作和执行相关的内部运算工作分开，并通过指令预取队列尝试让取指与执行重叠。这个案例说明：低时延并不只来自某个门更快，

也来自**把等待路径重叠**。不过预取队列不能消除所有等待；分支改变控制流、总线访问受阻或指令消费速度超过预取速度时，执行仍会遇到空洞。

80386 的案例则说明另一件事：晶体管增加以后，芯片不只是把原有电路做得更快，还会把更多系统功能搬到片内。32-bit 数据通路、保护机制、地址转换和更复杂控制逻辑，都意味着更多组合判断、更多寄存器边界和更复杂的关键路径管理。处理器越复杂，越不能把“时延”只理解成单个门的传播时间；控制路径、数据路径、存储路径和外部接口路径都要分别分析。

四、门网络构成组合逻辑

组合逻辑的特点很直接：输出只由当前输入决定，不保存过去。它像一个硬件函数，但这个函数不是一步一步执行的过程，而是一张由门和连线组成的网络。输入变化后，信号沿多条路径同时传播；等延迟过去，输出稳定到当前输入对应的值。

设计组合逻辑时，可以把前一节的五步具体化为一张工作表。该工作表不要求一开始写出最简表达式，而是先把需求完整展开，再逐步收缩为硬件网络。

设计动作	产物	验收问题
列输入输出	信号表	每个 0/1 含义是否明确
列行为	真值表或条件表	是否覆盖所有输入组合
写证据	输出为 1 或 0 的条件	每条证据是否对应可实现路径
合成网络	基本门、选择器和译码器	是否有明确驱动者和选择者
检查物理限制	延迟、扇出、毛刺、电平余量	后级何时使用这个输出

先做一个三传感器判断器。输入 A、B、C 表示三个传感器，要求至少两个传感器为 1 时输出 1。这个需求不能只凭直觉画线，先列出行为：

A	B	C	Y	原因
0	0	0	0	一个条件都没有
1	0	0	0	只有一个条件
0	1	0	0	只有一个条件
0	0	1	0	只有一个条件
1	1	0	1	A 和 B 同时满足
1	0	1	1	A 和 C 同时满足
0	1	1	1	B 和 C 同时满足
1	1	1	1	三个都满足

从表中可以直接得到一个门网络：

$$Y = (A \text{ AND } B) \text{ OR } (A \text{ AND } C) \text{ OR } (B \text{ AND } C)$$

这条表达式里有三条“证据路径”。第一条路径证明 A、B 同时为 1；第二条路径证明 A、C 同时为 1；第三条路径证明 B、C 同时为 1。最后的 OR 不是任意合并，而是在检查三条证据中是否至少有一条成立。按这种方式阅读，门网络就不只是公式变形，而是把真值表中每一种输出为 1 的原因接成硬件。

两个 bit 相加是第二个基本组合逻辑例子。输出不止一个 bit，因为 1+1 得到二进制 10，需要一个和位 S 和一个进位 C。

A	B	S	C
0	0	0	0
0	1	1	0
1	0	1	0

1	1	0	1
---	---	---	---

观察表格可以看到，S 在 A、B 不同时为 1，C 在 A、B 同时为 1：

```
S = A XOR B
C = A AND B
```

这就是半加器。它已经完成了一位相加，但它还缺一个输入：来自低位的进位。只要要做多位加法，最低位之外的每一位都不能只看 A 和 B，还要看低一位是否进上来。

全加器输入为 A、B、Cin，输出为 S、Cout。可以先把 A 和 B 用半加器相加，得到临时和 S1 和进位 C1；再把 S1 和 Cin 用第二个半加器相加，得到最终和 S 和第二个进位 C2；最后把两个进位合并。

```
S1 = A XOR B
C1 = A AND B
S = S1 XOR Cin
C2 = S1 AND Cin
Cout = C1 OR C2
```

把全加器一位一位串起来，就得到串行进位加法器。它的逻辑很清楚：第 0 位算出进位，传给第 1 位；第 1 位再把进位传给第 2 位。问题也很清楚：若最低位产生的进位一路传到最高位，最高位必须等前面所有进位稳定后才能稳定。这是组合逻辑里第一次真正遇到“逻辑正确”和“速度足够”的差别。真值表保证最后结果正确，传播延迟决定结果多久之后才可靠。

全加器也可以用生成与传播来阅读。若 A 和 B 同时为 1，本位一定产生进位，称为生成；若 A 和 B 恰好一个为 1，那么本位不会自己产生进位，但会把输入进位传到输出，称为传播。

```
generate = A AND B
propagate = A XOR B
Cout = generate OR (propagate AND Cin)
S = propagate XOR Cin
```

这种写法为后面的加法器优化留下接口。串行进位把进位逐位传递，结构简单但最长路径随位数增长；更快的加法器会提前计算若干位的生成和传播关系，减少等待层数。本章只要求读者看清一个事实：组合电路不仅有“算什么”，还有“最慢路径从哪里穿过”。算术电路尤其如此，因为进位路径常常决定 ALU 的速度上限。

把全加器再向前推进一步，就得到 1-bit ALU 切片的雏形。ALU 不是不可分解的黑箱，而是许多位宽相同的切片并排工作：每一位接收本位的 A、B、来自低位的进位，以及操作选择信号；它同时准备若干候选结果，例如 AND、OR、XOR 和 ADD；最后由选择器决定哪一种候选结果成为本位输出。进位输出则继续传给高一位或交给更快的进位逻辑处理。

```
and_result = A AND B
or_result = A OR B
xor_result = A XOR B

sum_result = A XOR B XOR Cin
Cout = (A AND B) OR ((A XOR B) AND Cin)

Y = mux(op, and_result, or_result, xor_result, sum_result)
```

这段行为式只是说明逻辑关系，不表示硬件按行执行。真实电路中，AND、OR、XOR 和加法候选路径可以同时传播；op 控制选择器，让其中一路成为输出。于是，一个 1-bit ALU 切片至少包含三类组合结构：基本逻辑门、全加器、选择器。多个切片并排以后，AND/OR/XOR 这类逐位运算的延迟主要取决于本位门和选择器；ADD 则还

受进位路径影响。

候选操作	本位需要的组合逻辑	关键验收点
AND	$A \text{ AND } B$	路径短，但仍受负载和选择器影响
OR	$A \text{ OR } B$	适合逐位屏蔽、合并标志位或控制位
XOR	$A \text{ XOR } B$	常用于差异检测、加法和翻转控制
ADD	全加器与进位链	最长路经常由进位传播决定
选择输出	op 控制的多路选择器	op 必须稳定，不能让错误候选短暂输出到危险后级

这个切片模型提前解释了 CPU 数据通路里的三个常见事实。第一，位宽增加不只是“多写几个 bit”，而是复制更多切片，并重新处理进位、布线和负载。第二，控制信号不是附属注释； op 、进位选择、结果写回选择都会直接改变哪条硬件路径有效。第三，ALU 的延迟不是单个门延迟，而是由候选路径、进位路径、选择器和输出负载共同决定。8086 的 16-bit 数据通路、80386 的 32-bit 数据通路，以及现代 64-bit 执行单元，都可以从这个切片观点继续放大的理解。

多路选择器解决“多个输入选一个输出”。2 选 1 选择器的行为是：

```
if sel = 0, Y = A
if sel = 1, Y = B
```

上述 if 只是描述行为，电路本身没有顺序执行。它的门级表达式是：

$$Y = (\text{NOT } sel \text{ AND } A) \text{ OR } (sel \text{ AND } B)$$

选择器的作用很基础。若一条输出线可能来自加法器，也可能来自传感器，也可能来自某个寄存器，那么必须有一个明确的选择信号决定哪一路接到输出。没有选择器，多路来源同时驱动同一条线就会冲突。

选择器的控制信号本身也需要验收。若 sel 悬空，输出可能在 A 和 B 之间随机变化；若选择信号来自毛刺严重的组合网络，输出可能短暂选择错误来源；若 A、B 属于不同电源域或不同稳定时间，选择器输出还会继承这些边界问题。因此，选择器不是“安全地把两根线接在一起”，而是“在一个可靠控制信号约束下，只让一路值成为输出”。

选择器检查项	问题	失败后果
选择信号默认值	上电或复位前选哪一路	输出来源不确定
数据输入稳定性	被选中的输入是否已稳定	选择了正确来源但值仍不可靠
未选输入行为	未选输入是否允许变化	不应通过内部耦合影响输出
输出负载	选择器是否能驱动后级	边沿变慢或电平不足

译码器做相反方向的事：把一个二进制编号展开成 one-hot 信号。2-bit 输入可以选中 4 条输出中的一条。

输入	输出
00	$Y0=1, Y1/Y2/Y3=0$
01	$Y1=1$ ，其他为 0
10	$Y2=1$ ，其他为 0
11	$Y3=1$ ，其他为 0

选择器把多路收成一路，译码器把编号展开成多路选择。后面无论是寄存器阵列、数据通路，还是存储器地址选择，都会反复看到这两个结构。

选择器还可以直接实现布尔函数。若一个函数有两个输入 A、B，可以用 A 作为选择信号，选择器的两个数据输入分别接“当 A=0 时 Y 应该等于什么”和“当 A=1 时 Y 应该等于什么”。例如 XOR 中，当 A=0 时，Y=B；当 A=1 时，Y=NOT B。因此一个 2 选 1 选择器加一个反相器即可实现 XOR：

```
if A = 0: Y = B
if A = 1: Y = NOT B
Y = mux(A, B, NOT B)
```

这种读法对后面的数据通路很重要。ALU 输入选择、写回选择、下一 PC 选择，本质上都可以看成“控制信号选择哪一路值成为输出”。选择器不是附属小部件，而是把多种候选硬件动作收束成一个当前动作的基本结构。

译码器则常用于“编号变成单独使能”。例如 2-bit 编号选择 4 个寄存器中的一个写入，译码器输出 one-hot 写使能：同一时刻只允许一个目标被写。若译码器错误地产生两个 1，就可能同时写两个寄存器；若所有输出都是 0，本周期写操作会消失。后面寄存器阵列和控制器都会反复依赖这个性质。

结构	典型用途	必须保证
选择器	多个来源选一个输出	同一时刻只有一路被选中成为结果
译码器	编号展开成 one-hot 使能	合法编号只激活一个目标
编码器	多个请求压缩成编号	多请求同时有效时要有优先级或报错
比较器	判断相等、大小或条件成立	输入解释方式必须明确

编码器和比较器也是常见组合电路。编码器把 one-hot 或优先级请求压缩成编号；比较器判断两个值是否相等，或者按某种解释比较大小。安全启动控制板可以用简单编码器把故障来源压缩成故障码：没有故障为 00，温度故障为 01，电流故障为 10，若两者同时故障则输出优先级最高的一项或输出“多故障”码。关键在于规则必须写清楚，不能让两个输入同时为 1 时输出处于未定义状态。

temp_alarm	current_alarm	故障码	说明
0	0	00	无故障
1	0	01	温度故障
0	1	10	电流故障
1	1	11	多故障或最高优先级故障

若系统选择优先级编码，就要把优先级写进规格。例如温度故障比电流故障优先，则两个报警同时为 1 时输出温度故障码；若系统更重视诊断完整性，则两个报警同时为 1 时输出多故障码。二者没有绝对优劣，但不能在电路里含糊。

编码策略	规则	适合场景
优先级编码	多个请求同时有效时选择最高优先级	中断控制、快速响应最重要的请求
多故障编码	多个请求同时有效时输出专用多故障码	故障诊断和维护记录
非法状态报警	非 one-hot 输入触发错误输出	希望尽早暴露编码器前级错误

比较器也必须说明数值解释。同样的 bit 模式，用无符号数解释和用补码有符号数解释，大小关系可能不同。以 4-bit 为例，1111 作为无符号数是 15；作为补码有符号数是 -1。若比较器输出 less_than，规格必须说明它比较的是无符号大

小、有符号大小，还是只比较 bit 模式是否相等。

比较类型	示例	说明
相等比较	<code>A == B</code>	只关心每一位是否相同，通常不涉及数值解释
无符号大小	<code>1111 > 0001</code>	<code>1111</code> 表示 15
有符号大小	<code>1111 < 0001</code>	<code>1111</code> 表示 -1， <code>0001</code> 表示 1
条件比较	报警级别是否超过阈值	必须说明编码范围和非法值处理

现在把贯穿案例写成一个完整组合网络。安全启动控制板至少需要两个输出：`fault_lamp` 和 `allow_start`。故障灯由报警证据决定；启动允许由启动按钮、安全条件和无故障共同决定。

```
fault_lamp = temp_alarm OR current_alarm
no_fault = NOT fault_lamp
allow_start = start_button AND cover_closed
               AND emergency_ok AND no_fault
```

这组表达式可以继续展开成门网络。第一层 OR 汇总故障证据；第二层 NOT 得到无故障证据；第三层多输入 AND 汇总启动按钮和安全条件。若硬件库里没有四输入 AND，可以用二输入 AND 逐级合成：

```
safe_chain = cover_closed AND emergency_ok
request_ok = start_button AND no_fault
allow_start = safe_chain AND request_ok
```

这种重写在逻辑上等价，但硬件上改变了门级分组。若 `cover_closed` 和 `emergency_ok` 来自同一个连接器，先合成 `safe_chain` 可能便于布线；若 `fault_lamp` 驱动多个后级，`no_fault` 前后可能需要缓冲。组合逻辑的表达式只是第一层结果，门级分组、负载和时序仍要继续检查。

内部信号	逻辑含义	硬件检查点
<code>fault_lamp</code>	任一报警成立	OR 路径是否能驱动指示灯和后级逻辑
<code>no_fault</code>	没有报警证据	是否由反相器恢复为强 0/1
<code>safe_chain</code>	保护盖和急停都满足	任一安全条件断开时是否默认禁止
<code>request_ok</code>	有启动请求且无故障	按钮抖动是否会传到后级
<code>allow_start</code>	所有条件满足	后级何时采样，是否存在毛刺风险
<code>motor_enable_cmd</code>	发给执行机构的使能命令	是否经过时序边界或专用安全链路

组合逻辑设计到这里仍然没有引入“步骤执行”。所有输入同时作用于网络，所有中间信号同时传播。若 `temp_alarm` 从 0 变 1，`fault_lamp` 会在 OR 路径延迟后变为 1，`no_fault` 会在反相器延迟后变为 0，`allow_start` 会在 AND 路径延迟后关闭。这个传播链条不是程序调用栈，而是多条电气路径的延迟叠加。

`motor_enable_cmd` 需要比 `allow_start` 更谨慎。前者是给外部执行机构的命令，后者只是内部组合判断。直接令 `motor_enable_cmd = allow_start` 在最小示例里看似合理，但正式系统通常会增加时序边界、复位保持、驱动级和独立停机链路。原因很简单：组合判断可以短暂过渡，执行机构不应响应这种过渡。

```
allow_start = start_request AND cover_closed
```

```
AND emergency_ok AND no_fault
```

```
motor_enable_cmd = registered_allow_start AND power_stage_ready
```

上式中的 `registered_allow_start` 表示已经在时钟边界保存过的允许结果，`power_stage_ready` 表示功率级自身已准备好并未报告故障。第一章还没有正式引入触发器，所以这里只把它作为边界预告：组合逻辑负责给出当前条件下的判断，执行机构命令通常还需要由时序逻辑和驱动电路接管。这样组织，能把“判断是否允许”和“真正驱动负载”分成两个可验收层次。

可以把安全启动控制板整理成一份组合规格。规格不必一开始就写门级细节，但必须把输入语义、输出语义和异常策略写清楚。这样做的好处是，之后无论用分立门、可编程逻辑，还是把同一逻辑放进更大的控制器，行为边界都保持一致。

规格项目	内容	设计约束
输入前提	输入已完成默认值、电平和消抖处理	原始触点不直接进入安全组合式
允许条件	启动请求、保护盖闭合、急停正常、无故障	任一条件缺失都不得允许启动
故障输出	温度或电流报警均点亮故障灯	同时报警要有明确编码策略
默认安全	未知、断线、非法编码倾向于禁止启动	化简逻辑不得破坏该策略
使用时刻	后级只能在信号稳定后使用	异步使能需要额外毛刺检查

下面给出这类小组合模块的完整书写顺序。它看起来比直接写一个公式更长，但每一步都对应一种常见错误：信号没定义、非法状态没处理、公式和需求不一致、实现后电气条件不满足。

顺序	书写内容	检查目标
1	列出原始输入、内部语义信号和输出	不把物理触点直接当作可靠 bit
2	说明每个信号的有效电平和默认值	避免低有效、断线和上电默认错误
3	写出正常行为和异常行为	不让非法组合默默进入允许状态
4	写逻辑表达式或真值表	让需求可机械检查
5	映射到门、选择器、译码器或编码器	明确驱动来源和选择关系
6	检查延迟、负载、毛刺、电平余量	证明实物能按时产生可靠输出
7	记录测量或仿真证据	让设计从文字规格落到可验证对象

把这七步用于 `allow_start`，可以得到更严格的文字规格：输入必须已经完成调理；`start_request` 为 1 表示一次经过确认的启动请求；`cover_closed` 为 1 表示保护盖闭合且传感器链路有效；`emergency_ok` 为 1 表示急停链路处于允许运行状态；`no_fault` 为 1 表示没有温度或电流报警；只有四者同时为 1，`allow_start` 才能为 1；任何未知、断线、未调理、非法编码或复位保持状态，都不应使 `allow_start` 为 1。

<code>allow_start</code> 条 必要证据 件	缺失时的结果
--------------------------------------	--------

start_request=1	按钮请求已经消抖或同步确 认	不启动
cover_closed=1	保护盖传感器明确证明闭合	不启动
emergency_ok=1	急停链路明确证明正常	不启动或停机
no_fault=1	无温度、电流等故障证据	不启动并报警
输出稳定	组合路径已越过最坏延迟	后级暂不采样或不驱动执行 机构

该表应按硬件合同阅读。若某个条件缺失却仍能启动，就是需求错误或实现错误；若输出尚未稳定就驱动执行机构，就是时序边界错误。组合逻辑的严肃性正在于此：它没有软件中的异常处理栈，只有电路在每一种输入情况下实际产生的电平。

为了让贯穿案例形成闭环，可以把安全启动控制板从外部物理输入一路写到执行命令。该表不是新增功能，而是把前面分散的原则合成一条可审查链路：原始线缆先进入电气边界，调理后得到内部语义信号，组合网络只处理已经可靠的 bit，危险输出再经过时序和驱动边界。

层次	典型对象	合同要求
原始输入	start_button_raw、传感器触点、报警线	允许抖动和噪声，但必须有默认路径、保护和极性说明
输入调理	上拉/下拉、滤波、施密特触发、电平转换	把板外现象整理成稳定内部电平，不让悬空和慢边沿直接进入核心
语义信号	start_request、cover_closed、emergency_ok	每个 0/1 含义清楚，低有效信号先转换为可审查语义
组合判断	fault_lamp、no_fault、allow_start	真值表覆盖正常、异常、非法和默认状态，化简不破坏安全策略
时序边界	registered_allow_start、复位保持、采样时刻	只在组合路径稳定后保存或使用结果，避免毛刺进入危险动作
输出驱动	motor_enable_cmd、指示灯驱动、功率级使能	具备足够驱动能力，上电和复位期间保持禁止，故障时进入安全状态
验收证据	原理图、数据手册、示波器、逻辑分析仪、测试矩阵	证明电平、延迟、负载、毛刺和异常输入处理都满足合同

这张闭环表也说明为什么第一章要同时讲电平、器件、门和组合逻辑。如果只看组合表达式，会把 start_request 当作天然可靠的变量；如果只看器件，又难以说明为何这些输入最终必须合成为一个 allow_start。正式硬件说明应当让两者互相校验：每个语义信号都能追溯到电气边界，每个组合输出都能追溯到真值表和路径证据，每个执行命令都能追溯到时序和驱动验收。

把同一思路放大到 8086 和 80386，就能看出早期处理器为什么需要大量组合模块。以 Intel 历史速查表所列参数为例，8086 于 1978 年引入，约 29,000 个晶体管，采用约 3 微米制造技术；Intel386 DX 于 1985 年引入，约 275,000 个晶体管，制造技术从约 1.5 微米进入后续 1 微米级别。这些数字本身不是本章重点，重点在于它们对应的结构变化：晶体管数量增加以后，可以容纳更宽的数据通路、更复杂的译码、更丰富的寄存器和更强的总线控制；工艺尺寸缩小以后，门和连线的寄生电容下降，单位距离连接更短，许多路径的延迟随之降低。

芯片	年代与规模	本章可观察的硬件变化	对时延的意义
8086	1978 年，约 29,000 晶体管	16-bit 数据通路、指令译码、总线接口	已经需要大量组合控制与寄存器边界配合

80386	1985 年，约 275,000 晶体管	32-bit 数据通路、更多片上逻辑减少外部更复杂保护和地址机制	协作等待，关键路径需重新组织
现代 CPU	数十亿级晶体管	多级缓存、乱序执行、预测、片上互连	不只让门更快，还减少取数和等待的次数

这里要避免一个误解：现代芯片低时延不是因为“逻辑门不再有延迟”。门仍然有延迟，连线仍然有电容，存储器仍然需要访问时间。现代芯片做的是把延迟**压低、隐藏、分层**和**局部化**。压低，是通过更小器件、更短路径、更强驱动和更好的门库实现；隐藏，是通过流水线、并行执行和预测让多个动作重叠；分层，是通过寄存器、L1/L2/L3 cache、片上缓冲和预取减少远距离访问；局部化，是把频繁通信的结构放得更近，避免长线和片外总线成为每次操作的必经路径。

五、延迟、负载、毛刺与验收

门电路在纸上像瞬间给出输出的函数，真实硬件却至少有四个限制。第一，输入变化后，输出需要传播延迟才稳定。第二，一个输出能驱动的后级输入数量有限，后级太多会让切换变慢，甚至让电平余量变差。第三，门的输出必须重新形成清楚的 0/1，不能只靠一个偏弱路径勉强改变节点电压。第四，多条路径延迟不同，输入变化过程中可能出现短暂毛刺。

限制	含义	后续影响
传播延迟	输出不会在输入变化的同一瞬间稳定	组合逻辑层数越多，稳定越慢
扇出	一个输出只能可靠驱动有限后级	大网络需要缓冲器和分层布线
电平恢复	输出要重新形成清楚 0/1	不能只追求逻辑表达式正确
毛刺	不同路径到达时间不同	后级不能在毛刺期间采样

考虑一个反例。若把十几个门的输入全接到同一个弱输出上，真值表仍然可以保持正确，但物理电路可能切换缓慢。输出从 0 变 1 时，要给所有后级输入电容充电；如果下一级在它尚未进入可靠高电平区间时读取，就可能看到旧值或不确定值。因此，分析组合逻辑时不能只数门的个数，还要考察最长路径、负载和稳定时间。

扇出问题可以放在安全启动控制板里理解。若 `fault_lamp` 只驱动一个小逻辑门，边沿可能很快；若它同时驱动指示灯缓冲、记录电路、禁止启动网络和外部连接器，等效负载明显增加。正确做法通常不是让一个弱节点直接驱动所有后级，而是分层缓冲：一个内部故障信号先驱动若干缓冲器，再由缓冲器分别驱动不同区域。缓冲器不改变逻辑含义，却改变驱动能力和负载隔离。

连接方式	可能结果	更可靠的处理
一个弱输出驱动多个后级	边沿慢，电平余量下降	增加缓冲或分层驱动
长线直接接到敏感输入	容易耦合噪声和振铃	加强驱动、终端处理或同步采样
逻辑输出直接驱动指示灯负载	负载过重，影响逻辑电平	使用驱动级隔离指示灯负载

短暂毛刺也很常见。假设一个输出由多条逻辑路径合成，输入同时变化时，每条路径稳定的时间可能不同。真值表只告诉我们变化前和变化后的稳定结果，却不保证中间不会短暂出现错误电平。如果后级只是另一个组合门，毛刺可能很快消失；如果后级在毛刺期间被时钟采样，错误就可能被保存下来。后面讲触发器和时钟时，会把这个问题放大讨论。

一个典型毛刺表达式是：

$$Y = (A \text{ AND } B) \text{ OR } ((\text{NOT } A) \text{ AND } C)$$

假设 $B=1$ 、 $C=1$ 。稳定状态下，不论 $A=1$ 还是 $A=0$ ， Y 都应为 1： $A=1$ 时第一项成立， $A=0$ 时第二项成立。问题出现在 A 翻转的瞬间。若 A 从 1 变 0，第一条路径可能先关断，而 $\text{NOT } A$ 经过反相器后才让第二条路径打开，中间可能出现一个短暂间隙，使 Y 从 1 短暂掉到 0。真值表看不见这个瞬间，因为真值表只描述稳定输入和稳定输出。

时刻	路径状态	Y 的风险
A=1 稳定	A AND B 成立	Y=1
A 正在翻转	第一条路径可能已关，Y 可能短暂为 0 第二条路径尚未开	
A=0 稳定	NOT A AND C 成立	Y=1

若这个 Y 只是驱动另一个组合网络，并且后级只在更晚的时钟边界采样，毛刺可能不会造成状态错误；若 Y 直接作为写使能、复位、片选或异步控制信号，短暂毛刺就可能触发真实动作。安全启动控制板里的 `allow_start` 尤其需要谨慎：即使最终会回到正确值，也不能让一个窄脉冲误开电机使能。

上面的毛刺可以用增加共识项来消除。表达式 $Y = (A \text{ AND } B) \text{ OR } ((\text{NOT } A) \text{ AND } C) \text{ OR } (B \text{ AND } C)$ 在 $B=1$ 、 $C=1$ 且 A 翻转时有静态 1 毛刺风险。加入第三项 $B \text{ AND } C$ 后， A 翻转期间仍有一条不依赖 A 的路径保持 Y 为 1：

$$Y_{\text{safe}} = (A \text{ AND } B) \text{ OR } ((\text{NOT } A) \text{ AND } C) \text{ OR } (B \text{ AND } C)$$

这不是为了改变稳定真值表。因为当 $B \text{ AND } C$ 为 1 时，原表达式在 $A=0$ 或 $A=1$ 下本来也为 1；新增项只是在过渡期间提供连续路径。代价是多一个门和更多负载。是否值得加入，取决于这个输出是否会直接驱动危险控制线，或者是否会在毛刺窗口内被采样。

危险信号需要分级处理。并非所有组合输出都必须做成完全无毛刺；但直接驱动写使能、片选、复位、时钟门控、功率开关或电机使能的信号，不能只用“最终会正确”作为验收标准。对这些信号，常见策略包括：把组合结果先送入触发器，在时钟边界后再使用；避免用毛刺风险高的组合式直接产生异步控制；必要时加入共识项或专用使能门；对外部执行机构再增加独立保护链路。

输出类别	毛刺容忍度	处理原则
普通数据位	通常可容忍短暂过渡	后级在稳定时钟边界采样
状态指示灯	一般可容忍短暂不可见闪动	关注驱动能力和人眼可见行为
写使能/片选	容忍度低	避免异步毛刺，必要时寄存后使用
复位/停机/电机使能	容忍度极低	需要安全默认、路径冗余和严格时序验收

组合路径还需要时间账本。假设某个输出要在 20ns 内稳定，而路径包含输入缓冲、两级门、一级选择器和输出缓冲，就要把最坏延迟相加。下面的数字只是说明账本方法，不代表某个工艺或芯片规格。

路径段	最坏延迟	说明
输入缓冲	2ns	外部信号进入内部逻辑
第一层门	3ns	例如报警 OR 或安全条件 AND
第二层门	4ns	例如 no_fault 与请求合并

选择器或缓冲	5ns	输出选择或驱动增强
布线和负载	3ns	走线、电容、扇出影响
合计	17ns	若要求 20ns 内稳定，则余量 3ns

若后续系统在 15ns 时就使用这个输出，稳定真值表仍然不够；输出可能还在传播或毛刺阶段。第二章要引入的触发器和时钟边界，正是为了规定“什么时候保存”。组合逻辑在边界之间准备结果，时钟边界只应采样已经稳定、满足建立时间要求的信号。

时间账本要使用最坏情况，而不是典型情况。典型延迟说明某些样品在某些条件下通常有多快；最坏延迟才说明在温度、电压、工艺和负载不利时是否仍能工作。若一条路径在实验室常温下 12ns 稳定，并不能证明它在低电压、高温、最大负载下也能在 15ns 内稳定。正式规格通常同时给出最小、典型、最大值；做时序验收时应以满足系统要求的最坏组合为准。

延迟读法	含义	使用方式
最小延迟	某路径最快可能多久变化	检查保持时间、窄脉冲和过早到达
典型延迟	常见条件下大致延迟	便于估算，不作为可靠验收边界
最大延迟	不利条件下最晚多久稳定	检查时钟周期和采样时刻是否足够
偏斜	不同路径或不同信号到达时间差	解释毛刺、竞争和采样风险

现在可以更系统地回答“现代芯片为什么时延低”。第一层原因来自器件和工艺。更小的晶体管通常意味着更小的栅电容、更短的沟道、更短的局部连接和更低的单个门负载；同样的逻辑函数，在更小、更近的器件中传播，很多局部路径的绝对时间会下降。但这不是无条件成立。长连线、全芯片时钟、供电网络和散热会逐渐成为新的限制，所以现代芯片必须同时优化**晶体管**和**连线**。

第二层原因来自片上集成。8086 时代，处理器与内存、外设和许多支撑逻辑之间存在大量片外通信；片外总线距离长、电容大、引脚数量有限，等待代价高。8086 已经用总线接口单元和执行单元的分工，以及指令预取队列，尝试把取指和执行重叠起来。到 80386，更多地址转换、保护和控制逻辑进入处理器内部，32-bit 数据通路和更丰富的片上控制减少了对外部协作的依赖。再往后，cache、浮点单元、内存控制器、图形或 I/O 控制器逐步进入同一颗芯片或同一封装，许多原本必须跨板级总线的路径被缩短为片上路径。

阶段	降低等待的做法	与本章概念的关系
8086	取指与执行尝试重叠，内部控制信号组织总线周期	组合控制和寄存器边界共同工作
80386	更宽数据通路、片上地址转换和保护机制	更多功能靠片上网络完成，减少外部协作
80486 以后	cache 和更多执行单元进入处理器	常用数据靠近核心，外部访问次数下降
现代 CPU/SoC	多级 cache、预测、预取、乱序、片上互连	把慢路径分层、隐藏或局部化

第三层原因来自存储层次。主存容量大，但距离核心远、访问路径长；寄存器和 L1 cache 容量小，却离执行单元近。现代芯片通过多级 cache 把最常用的数据和指令放在更近的位置。一次 L1 命中不是因为 DRAM 变得同样快，而是因为本次访问根本没有走到 DRAM。时延降低来自“常见情况走短路径”，不是“所有路径都同样

短”。

位置	硬件特点	对时延的影响
寄存器	位于执行核心内部，数量少	最靠近 ALU，供数据通路快速使用
L1 cache	小而近，面向当前核心	常用指令和数据低时延命中
L2/L3 cache	更大但更远，可能被多个核心共享	减少访问主存的频率
DRAM	容量大，位于芯片外或封装外路径更远	未命中时延高，需要缓存、预取和并行隐藏
存储设备	比主存更慢，通常通过 I/O 层访问	不能直接参与普通指令级低时延路径

第四层原因来自流水线和并行。把一条很长的组合路径拆成多个较短阶段，每个阶段之间放入寄存器，就能提高时钟频率。这样做降低了每个时钟周期需要容纳的组合延迟，但并不等于每条指令的总周期数一定减少。现代处理器还会用分支预测、乱序执行、多个执行单元和内存预取，把将来可能需要的工作提前准备，把等待主存或分支结果的空洞尽量填满。它们降低的是常见执行路径上的可见等待。

机制	降低哪类等待	代价或限制
流水线	单个周期内的组合路径过长	分支错误会冲刷流水线
分支预测	等待条件跳转结果	预测错误带来回滚成本
乱序执行	等待某个慢操作时执行其他就绪操作	控制逻辑复杂，功耗和验证成本高
预取	等待未来内存访问	预取错误会浪费带宽和 cache 空间
多级 cache	等待远距离主存	cache 未命中仍然昂贵
片上互连与近数据布局	等待长线和片外总线	面积、布线和一致性协议更复杂

因此，现代芯片的低时延是一组工程选择的结果。器件缩小降低局部门延迟；门级库、缓冲和布局缩短关键路径；寄存器边界把长组合逻辑切成阶段；cache 把常用数据放近；预测和乱序把等待隐藏；片上集成减少跨芯片通信。若某次访问落到 **DRAM**、分支预测失败、cache 未命中或跨核心通信，时延仍会显著上升。严肃的硬件分析必须同时说明**最快常见路径**和**最坏慢路径**。

现代 CPU 的一个典型低时延路径是：操作数已经在寄存器或 L1 cache 中，分支预测正确，所需执行单元空闲，结果能在相邻流水线阶段之间传递。此时，芯片用局部电路、短路径和寄存器边界把等待压到很低。相反，若数据不在 cache 中、分支预测错误或需要跨核心同步，同一条指令流会进入慢路径。现代芯片的工程目标不是让所有路径同样快，而是让高频路径短、让慢路径少发生，并在慢路径发生时尽量隐藏等待。

还要区分**时延**和**吞吐率**。流水线可以让每个周期都有不同指令处在不同阶段，从而提高吞吐率；但一条具体指令从进入流水线到完成，仍然要经过多个阶段。多执行单元可以同时处理多条互不依赖的操作，但无法让一条有严格依赖链的操作跳过必要计算。cache 可以显著降低常见访问时延，但不能让未命中的访问变成命中。理解现代芯片时，必须同时问两个问题：单次操作最快多久返回，连续大量操作每单位时间能完成多少。

还要区分反相器和缓冲器。反相器改变逻辑含义，缓冲器通常保持逻辑含义但增强驱动能力或隔离负载。很多时候，插入缓冲器不是为了“多一个门”，而是为了让一

个信号能推得动更长的线或更多的后级。软件读者容易觉得保持值不变的缓冲器没有意义，但在硬件里，驱动能力本身就是功能的一部分。

最后还要检查扇入。扇出关心一个输出驱动多少后级，扇入关心一个门接收多少输入。四输入、八输入甚至更多输入的门在逻辑表达式里写起来很自然，但在硬件实现中可能被拆成多级结构；输入越多，门内部路径、电容和延迟也可能越大。把 `allow_start` 写成一个五输入 AND 并不意味着实际芯片里一定存在一个理想五输入 AND。实现可能把它拆成树形结构，也可能先合成若干中间信号。拆分方式会改变最长路径和毛刺形态。

```
flat idea:
    allow_start = A AND B AND C AND D AND E

tree implementation:
    left  = A AND B
    right = C AND D
    mid   = left AND right
    allow_start = mid AND E
```

树形实现通常比线性串接更平衡，但会引入中间节点和不同路径长度。若 `E` 很晚才稳定，把它放在最后一级可能有利于减少无效切换；若 `A`、`B` 来自同一连接器，先合成 `left` 可能便于局部布线。这里没有脱离功能的“最佳写法”，只有在功能、负载、延迟、布局和安全策略之间作出的实现选择。

组合逻辑的最终验收，不应只写“真值表正确”。至少要检查以下项目：

验收项	检查内容	失败后果
功能	稳定输入下输出是否符合真值表	逻辑行为错误
默认状态	悬空、断线、复位前输入如何解释	异常时误启动或误使能
驱动能力	每个输出能否带动实际负载	电平被拖低、边沿过慢
路径延迟	最长组合路径是否能及时稳定	后级采到旧值或过渡值
毛刺风险	多路径汇合是否产生危险窄脉冲	写使能、复位、片选被误触发
低有效信号	信号命名和真值表是否一致	使能/禁止方向接反
物理边界	长线、外部输入、按钮抖动是否处理	干扰和抖动进入逻辑核心

对安全启动控制板，可以把验收写成矩阵。矩阵的价值在于把“正常输入”“边界输入”“异常输入”和“时间条件”放在一起，防止只验证最容易通过的几行真值表。

验收类别	要施加的条件	期望结果
正常允许	请求有效、盖闭合、急停正常、无故障	<code>allow_start=1</code> ，执行命令等待时序边界
正常禁止	任一安全条件为 0	<code>allow_start=0</code>
故障禁止	温度或电流报警为 1	<code>fault_lamp=1</code> ， <code>allow_start=0</code>
断线默认	保护盖、急停或报警链路断开	进入禁止或报警一侧
上电过程	电源未稳或复位保持	<code>motor_enable_cmd=0</code>
按钮抖动	原始按钮快速断续	内部请求不产生多次启动事件
最坏负载	故障灯、后级逻辑和连接器同时连接	电平仍满足后级阈值
最坏时序	输入在不利延迟组合下变化	后级只在稳定后采样或使用

常见失败也应明确记录。许多硬件问题并不是因为设计者不知道 AND、OR、NOT，而是因为把抽象层次接错：把原始输入当内部信号，把典型延迟当最坏延迟，把保留状态当不可能，把内部判断直接接到危险执行机构。

失败类型	表面现象	根本原因
悬空输入	偶尔误启动或误报警	没有默认上拉/下拉或输入调理
低有效误读 电平不兼容	复位、片选或急停方向相反 单独测试正常，接入后失效	信号合同未先于公式建立 前后级阈值和驱动能力不匹配
毛刺误触发 扇出过大 时序乐观 不关心误用	最终值正确但出现短促动作 边沿慢，高电平不够高 常温可用，高温或低压失效 异常状态下进入允许输出	危险输出直接使用组合信号 一个输出承担过多负载 用典型值代替最坏值验收 把可能发生的故障组合当作无关项

若要把本章内容带入后续 CPU 数据通路，读者应保留两种同时成立的视角。第一，抽象视角：逻辑门实现布尔函数，组合网络实现加法、选择、译码和比较。第二，机器证据视角：每个输出都来自具体电流路径，每条路径都有延迟、负载和阈值，每个危险输出都需要明确的使用时刻。缺少前者，无法构造大系统；缺少后者，系统只是在纸上正确。

到这里，本章完成了从电平到组合逻辑的闭环：节点要有明确驱动路径；电平要有噪声余量；二极管提供方向性但不能恢复电平；受控开关让信号控制电流路径；路径表和真值表共同定义门；门网络构成加法器、选择器、译码器等组合电路；组合电路虽然没有记忆，却有延迟、负载和毛刺。下一章开始讨论另一件事：怎样让硬件保存状态，并按时钟边界一步一步推进。

章末练习与验收任务

本节练习要求使用文字、真值表、路径表和验收表完成。重点不是记忆门符号，而是证明某个 bit、某条路径、某个组合输出在真实硬件条件下可靠成立。

任务	要完成的产物	验收标准
按钮输入	为 <code>start_button_raw</code> 写出上拉或下拉方案、有效电平、断线默认值和消抖策略	松开、按下、抖动、断线四种状态都有明确内部语义
电平合同	给出一组前级 V_{OH}/V_{OL} 与后级 V_{IH}/V_{IL} ，计算高低噪声余量	说明余量为正、为零、为负时分别意味着什么
路径表	任选 NAND 或 NOR，列出四行真值表，并为每一行写出上拉与下拉路径状态	不出现稳定输入下的悬空或强冲突
低有效信号	将 <code>reset_n</code> 、 <code>chip_select_n</code> 或急停链路转换为正逻辑语义信号	说明原始电平、内部语义和默认安全值之间的关系
故障汇总	为 <code>temp_alarm</code> 与 <code>current_alarm</code> 设计故障灯和故障码逻辑	单故障、双故障、无故障和非法输入都有定义
1-bit ALU	写出 AND、OR、XOR、ADD 四类候选结果和 <code>op</code> 选择规则	区分逐位逻辑路径和加法进位路径的延迟来源
毛刺分析	分析 $Y = (A \text{ AND } B) \text{ OR } ((\text{NOT } A) \text{ AND } C)$ 在 $B=1$ 、 $C=1$ 、 A 翻转时的风险	能说明共识项为什么不改变稳定真值表，却改变过渡行为
安全闭环	按“原始输入、输入调理、语义信号、组合判断、时序边界、输出驱动、验收证据”重写安全启动控制板规格	每一层都能追溯到上一层，危险输出不能直接依赖未验收组合结果

芯片案例	分别用 7400、4000 CMOS、8086、80386 和现代 CPU 解释本章一个概念	每个案例都要指出对应的电平、路径、组合逻辑或低时延机制
测量计划	为按钮输入或故障输出写出万用表、示波器、逻辑分析仪各自要证明的内容	不把静态电压测量误认为完整时序和毛刺验收

完成这些练习后，读者应能把一个组合电路从自然语言需求推进到可验收规格：先定义信号含义，再列真值表和路径表，然后检查电平、负载、延迟、毛刺、默认状态和测量证据。这个顺序将直接用于后续寄存器、数据通路、控制器和整机接口。

本章的验收标准不是“会背 AND、OR、NOT、XOR 的真值表”，而是能把每个抽象还原成机器证据。

- 电平证据：前一级的最差高/低输出，必须落在后一级输入阈值允许的范围内，并留下噪声余量。
- 路径证据：任何输出节点都要能写成“高电平路径是否导通、低电平路径是否导通、是否存在冲突电流、是否存在悬空”四项。
- 门级证据：任选一个二输入门，列出 4 行真值表，再为每一行写出上拉网络和下拉网络状态。
- 组合证据：半加器的 Sum 是 XOR，Carry 是 AND；全加器是在同一张真值表中纳入进位输入后的组合电路。
- 延迟证据：同一个逻辑函数可以有不同门级网络，最长路径不同，稳定时间也不同。
- 负载证据：一个门的输出不是无限共享变量，后级数量、线长和缓冲策略会改变电气行为。
- 边界证据：外部输入、上电复位、开漏共享线、三态总线和电源域转换都必须先通过边界合同，才能进入普通组合逻辑。
- 安全证据：内部允许判断不等同于执行机构命令，危险输出必须说明默认状态、使用时刻和毛刺处理策略。
- 芯片案例证据：7400/4000 系列展示标准门芯片的电气合同；8086/80386 展示组合控制和数据通路如何扩展成处理器；现代 CPU 展示 cache、流水线、预测和片上集成如何降低常见路径的可见等待。
- 检查题：若组合网络输出短暂从 0 跳到 1 又回到 0，而最终值正确，它是否一定构成设计错误？答案取决于后级是否在毛刺期间采样。

经典系统教材常把抽象位函数和实际机器证据放在同一页上看。只看位函数会漏掉电平、驱动和时序；只看器件又会失去逻辑行为。两者合在一起，才是硬件设计对象。

状态、时钟与同步边界

第一章讲的是组合逻辑：输入稳定后，输出由当前输入唯一决定。它能计算，却不能保存过去。真正的机器必须能把某个结果留下来，等下一个步骤继续使用；也必须能规定“什么时候更新状态”，否则反馈路径会让硬件在一个周期内连续变化，机器无法分步骤推进。

本章将反馈、锁存器、触发器、时钟、复位、寄存器、计数器和状态机合并讨论。读完后，应当能把“状态”理解成真实硬件边界：一些 bit 在时钟边界被提交，边界之间由组合逻辑准备下一值。

核心思想

组合逻辑负责准备下一值，状态元件负责在边界保存下一值。同步硬件就是把这两件事反复组织起来。

一、反馈、锁存器与保存一个 bit

保存一个 bit 的起点是反馈。两个反相器首尾相接，会形成两个稳定状态。若左边节点为 0，右边经过反相后为 1；右边的 1 再反相回来，正好维持左边为 0。反过来，左边为 1、右边为 0 也能稳定。只要没有外部强制改变，这个状态会保持。

该结构说明，“保存一个 bit”对应电路中某些节点在反馈作用下停留于稳定物理状态。一个稳定状态解释为 0，另一个稳定状态解释为 1。状态的本质，是电路有能力在输入不再变化时仍然维持某个内部结果。

但单纯反馈还不足以形成可用存储单元。若无法可靠地把它从 0 改成 1，或从 1 改成 0，它只是一个稳定反馈环。因此需要写入控制。锁存器可以理解为带写使能的 1-bit 存储单元：写使能有效时，输入 D 可以影响输出 Q；写使能无效时，反馈通路维持原来的 Q。

写使能	数据输入	输出行为
有效	0 或 1	输出跟随输入变化
无效	任意	输出保持原值

例如一个“按住才能设置”的电路中，enable 为 1 时，D=1 会把锁存器写成 1；enable 回到 0 后，即使 D 变回 0，Q 仍保持 1。这样，过去某一时刻的输入就被硬件保存下来了。

锁存器的问题在于透明窗口。只要 enable 有效，输入变化就可能一路穿透到输出。若多个锁存器共用同一个 enable，并且它们之间还接着组合逻辑，状态可能在一个打开窗口内穿过多级，边界变得模糊。对小电路来说，锁存器易于分析；对大规模同步机器来说，更清晰的做法是把写入压到离散时钟边界。

二、触发器、时钟周期与采样窗口

触发器解决的是“什么时候写入”的问题。它不像锁存器那样在一段时间内透明，而是在时钟沿附近采样输入，把采样值保存到输出。

```
on clock edge:
  Q = D
between clock edges:
  Q holds
```

这让大电路有了清楚节奏。一个时钟沿之后，上一批触发器输出新状态；这些状态经过组合逻辑传播，形成下一批输入；到下一个时钟沿，目标触发器再统一采样。于是“计算”和“保存”被分开了。

最常见的同步路径可以写成：

```
source register -> combinational logic -> destination register
```

时钟沿到来后，源寄存器输出旧状态对应的新值。这个值进入组合逻辑，经过门延迟和布线延迟，最终到达目标寄存器输入。下一个时钟沿到来时，目标寄存器采样这个结果。因此频率不是越高越好。频率提高意味着周期缩短，留给组合逻辑稳定的时间变少。若周期短到最慢路径来不及稳定，目标寄存器就可能采到旧值、过渡值或错误值。

触发器采样也不是数学瞬间。输入 D 必须在时钟沿到来前稳定一小段时间，这称为建立时间；时钟沿之后还要继续稳定一小段时间，这称为保持时间。

约束	含义	违反后果
建立时间	采样前输入要提前稳定	采到错误值或不稳定值
保持时间	采样后输入要继续稳定	新旧值混入采样窗口
传播延迟	上一级输出和组合逻辑需要时间	周期太短会来不及

一条路径的周期账本可以写成：

```
clock period >= source clock-to-Q
                + combinational delay
                + wire delay
                + destination setup time
```

保持时间看起来更小，却同样危险。若源寄存器输出太快、路径太短，目标寄存器可能在同一个时钟沿之后立刻看到新值，从而破坏它本应采旧值的窗口。长路径会限制最高频率，短路径也可能破坏保持约束。

三、关键路径、复位与跨边界输入

一块同步电路里会有很多寄存器到寄存器路径。最长、最慢、最限制周期的那条叫关键路径。它可能是一串加法器，也可能是多级选择器，也可能是远距离布线加上复杂译码。

例如一个 32-bit 串行进位加法器接在两个寄存器之间。最低位进位要一位一位传到最高位，最高位结果最后才稳定。如果把这个加法器放在一个周期里完成，那么周期至少要等最坏进位链走完。若把运算拆成更短的几段，中间插入寄存器，单个周期可以变短，但完成一次完整运算需要更多周期。

结构选择	好处	代价
一周周期完成长运算	控制简单，单条动作周期数少	关键路径长，频率低
拆段插入寄存器	单周期路径短，频率可能高	完成一次动作需要更多边界

触发器通电后的值不一定符合设计预期。若一组状态寄存器随机落在不同值，后面的组合逻辑可能立刻进入非法状态。复位信号的作用，是把关键状态单元带到已知点，例如计数器清零、状态机回到 idle、控制寄存器进入安全默认值。

复位本身也要讲边界。若复位释放时，有的触发器先退出复位，有的后退出复位，状态机可能看到一半新状态、一半旧状态。常见处理是异步拉入复位以便快速进入安全状态，再同步释放，让所有相关触发器在清楚的时钟边界恢复工作。复位不是

“清零所有东西”这么简单。哪些状态需要复位，哪些数据寄存器可以不复位，哪些输出在复位期间必须安全，都要由硬件需求决定。

跨时钟或来自外部世界的输入还会带来亚稳态。若输入 D 正好在时钟沿附近变化，触发器内部节点可能短时间落在不稳定区域，既不像明确 0，也不像明确 1。多数情况下，它会在一小段时间后落回 0 或 1，但落回哪边、需要多久，不能按普通组合逻辑精确保证。因此来自另一个时钟节奏或外部世界的信号，不能直接拿来控制大面积电路，通常要先经过同步结构，降低亚稳态扩散概率。

四、寄存器、计数器与移位结构

一个触发器保存 1 bit。把多个触发器并排，并让它们共享同一个时钟边界，就得到多 bit 寄存器。寄存器可以理解为一组共享时钟和写使能的触发器。若写使能有效，在时钟沿采样输入；若写使能无效，保持原值。

```
on clock edge:
    if write_enable:
        Q = D
    else:
        Q holds
```

寄存器的意义不只是“能存数”。它把连续传播的组合逻辑切成离散步骤。上一个周期的结果在寄存器输出端稳定存在，成为本周期组合逻辑的输入；本周期算出的结果，要到下一个时钟沿才成为新的状态。共享同一提交边界是关键：寄存器保存的是同一个边界上的一组 bit，不是一位一位随意变化。

计数器的下一值来自当前值加一：

```
next = current + 1
on clock edge:
    current = next
```

该结构里有反馈，但反馈不是直接绕回去，而是经过加法器，并且被时钟边界切开。若 current 为 3，本周期组合逻辑算出 next 为 4；只有下一个时钟沿到来，寄存器才保存 4。随后组合逻辑再从 4 算出 5。如果没有寄存器边界，current+1 的结果会立刻回到输入，再加一，再回到输入，电路无法表达“每个周期只增加一次”。

移位寄存器把一组 bit 向左或向右移动。它可以用于串行输入、串行输出、简单延迟线和位级变换。

```
next[3] = current[2]
next[2] = current[1]
next[1] = current[0]
next[0] = serial_in
```

假设 current 为 1010，serial_in 为 1。下一个时钟沿后，寄存器变成 0101。注意，四个位不是按顺序一个一个搬。组合逻辑同时准备好四个 next 输入，时钟沿到来时一起写入。该例说明状态更新不一定是加法，也可以是重新排列、选择、清零、保持或装载外部输入。

五、状态机把硬件动作切成阶段

有限状态机由三部分组成：当前状态、输入条件、下一状态逻辑。当前状态保存在寄存器里，组合逻辑根据当前状态和输入条件决定下一状态。

以一个三阶段硬件控制器为例：

当前状态	条件	下一状态
IDLE	start=1	LOAD
LOAD	ready=1	RUN

RUN	count_done=1	DONE
DONE	clear=1	IDLE

若当前状态是 IDLE 且 start=0, 状态保持 IDLE; 若 start=1, 下一状态逻辑准备 LOAD, 等时钟沿到来后状态寄存器才真正变成 LOAD。状态机因此不会在一个周期里连续越过 IDLE、LOAD、RUN、DONE, 它每次只跨过一个明确边界。

状态机输出可以只由当前状态决定, 也可以同时依赖输入。前者输出更稳定, 后者反应更快但更容易受输入毛刺影响。基本原则是: 状态机不是若干状态名称的集合, 而是“状态寄存器 + 下一状态组合逻辑 + 输出组合逻辑”的硬件结构。

现在硬件已经能够保存一组 bit、按周期加一、按周期移动、按条件切换阶段。下一步要把这些能力用于更大的动作组织: 哪些寄存器在这个周期写入, ALU 做哪种运算, 哪一路数据被选择, 下一阶段去哪里。这就是控制器的基础。

本章的标注对象是“边界”和“状态转移表”。只说某个值会变化不够, 必须说明哪个时钟边界提交, 提交前后的组合路径是什么。

- 纸上实验: 画出一个 D 触发器前后各接一段组合逻辑, 标出当前周期的旧 Q、组合路径上的 D、下一个时钟沿后的新 Q。
- 时序证据: 为一条路径写出 $T_{clk} \geq t_{CQ} + t_{comb} + t_{setup} + t_{skew}$, 再解释每一项对应哪个硬件现象。
- 复位证据: 复位要让控制状态进入已知安全点, 但释放复位也要有同步边界, 不能让状态机半边先跑。
- 状态证据: 寄存器是一组并排触发器; 计数器是寄存器输出经过加法器反馈到输入; 移位寄存器是固定连线选择了相邻 bit。
- 控制证据: 状态机输出可以只依赖状态, 也可以依赖状态和输入; 后者反应快, 但更容易把输入毛刺传给控制线。
- 检查题: 若状态编码里出现未使用编码, 电路上电或受干扰进入它时, 下一状态逻辑应该怎样让机器回到安全状态?

经典系统教材通常会把组合逻辑和状态逻辑分开建模: 组合逻辑计算下一值, 状态元件在边界保存下一值。后面 CPU 的 PC、寄存器、状态机和控制器都遵守这个模型。

数值、ALU、数据通路与控制 器

前两章已经解决两个基础问题：bit 如何由电平和开关路径产生，状态如何由触发器和时钟边界保存。本章把第三部分压成一个厚章：固定宽度 bit 如何解释成数，算术和逻辑如何进入 ALU，值如何沿数据通路流动，控制器如何在每个周期给出一组一致的控制信号。

本章的重点是说明 CPU 出现之前的关键部件及其连接方式。到章末，读者应当能走读一条硬件动作：从哪些寄存器读旧值，经哪些选择器进入 ALU，结果如何带着标志位输出，最后由控制器决定是否写回状态。

一、固定宽度 bit 的数值解释

硬件只保存 bit。所谓整数、正负号、溢出，都是对 bit 模式的解释规则。若不先讲清楚表示，后面的加法器、减法器、比较器和 ALU 就会像在做抽象数学；事实上它们处理的始终是固定宽度的一组电线。

4-bit 无符号数 `b3 b2 b1 b0` 的值是：

```
value = b3*8 + b2*4 + b1*2 + b0
```

因此 `1011` 表示 11，因为它等于 $8+0+2+1$ 。无符号数没有负数概念，所有 bit 都贡献非负权重。4-bit 能表达的范围是 0 到 15，8-bit 能表达的范围是 0 到 255。位宽是硬件限制，不是显示格式。一个 4-bit 加法器只能输出 4 个结果 bit，再加上可能的进位。若 15 加 1，低 4 位回到 `0000`，最高位外产生一个进位。这个进位是无符号运算是否超出范围的证据。

补码表示让负数也能走同一个加法器。以 4-bit 为例，最高位不再只是 8，而是解释成负权重 -8：

```
value = b3*(-8) + b2*4 + b1*2 + b0
```

于是 `1111` 的值是 $-8+4+2+1=-1$ ，`1110` 的值是 -2，`1000` 的值是 -8。4-bit 补码范围是 -8 到 7。求一个数的相反数，可以按“取反加一”理解：

```
x = 0011 // 3
~x = 1100
~x + 1 = 1101 // -3 in 4-bit two's complement
```

这个规则适合硬件，因为取反可以用一排 NOT 或 XOR 门完成，加一可以交给加法器。于是 $a-b$ 可以变成：

```
a - b = a + (NOT b) + 1
```

同一组 bit 可以按无符号解释，也可以按补码解释。因此溢出判断也不同。

解释方式	关注点	例子
无符号	是否产生最高位外的进位	$15 + 1$ 超出 4-bit 无符号范围
补码有符号	正正得负或负负得正	$7 + 1$ 得到 <code>1000</code> ，被解释为 -8

以下 4-bit 加法给出同一结果在两种解释下的差异：

```
0111 + 0001 = 1000
```

若按无符号解释，这是 $7+1=8$ ，仍在表示范围内。若按补码解释，0111 是 7，0001 是 1，结果 1000 是 -8，正数加正数却得到负数，所以发生有符号溢出。硬件不会替使用者决定哪种解释成立；它只提供证据：结果 bit、进位、符号位变化、溢出标志。

移位器也是算术部件。左移一位通常相当于乘以 2，但前提是移出去的 bit 没有携带需要保留的信息。右移更容易出差别。逻辑右移在高位补 0；算术右移保留符号位，用于补码有符号数。

```
logical right shift: 1000 >> 1 = 0100
arithmetic right shift: 1000 >> 1 = 1100
```

若 1000 按 4-bit 补码解释为 -8，算术右移得到 1100，也就是 -4；逻辑右移得到 0100，也就是 4。两个结果差异巨大。原因不是移位器“出错”，而是控制信号选择了不同硬件规则。

二、ALU 把候选计算收进一个组合核心

ALU 是算术逻辑单元。它不是孤立出现的大型部件，而是因为加法、减法、按位逻辑、移位和比较反复出现，硬件需要把这些能力集中到一个受控制的组合逻辑块中。ALU 的本质是：多个候选计算电路并行存在，控制信号选择本周期采用的结果。

一个最小 ALU 可以写成：

```
ALU(a, b, op) -> result, flags
```

a 和 b 是两个操作数，op 是选择信号，result 是结果，flags 是结果的附带证据。

op	运算	硬件来源
ADD	$a + b$	加法器
SUB	$a - b$	加法器加反相控制
AND	$a \& b$	按位 AND 门阵列
OR	$a b$	按位 OR 门阵列
XOR	$a \text{ xor } b$	按位 XOR 门阵列
SHIFT	移位	移位器

ALU 里最值得细看的是 SUB。若单独做减法器，会增加面积和连线。补码让它复用 ADD 的加法器：

```
if op = ADD:
    b_to_adder = b
    carry_in = 0
if op = SUB:
    b_to_adder = NOT b
    carry_in = 1
result = a + b_to_adder + carry_in
```

实际电路里，b 的每一位前面可以接 XOR 门：当 sub 控制为 0，b 原样通过；当 sub 控制为 1，b 被取反。最低位进位输入也由 sub 控制。这样 ADD 和 SUB 共用同一条进位链。这个案例展示了硬件设计的常见思路：先找到数学规则之间的共用部分，再用少量控制信号复用同一块组合逻辑。

ALU 内部可以并行产生多个候选结果，再用多路选择器按 op 选出最终 result。

```
add_result
and_result
```

```

or_result
shift_result
result = mux(op, add_result, and_result, or_result, shift_result)

```

这不表示所有运算都“按顺序算一遍”。组合逻辑是并行传播的：加法器、按位逻辑、移位器都可能同时产生自己的候选输出。最终哪一路被后级看见，由 mux 决定。代价也随之出现。候选电路越多，ALU 面积越大；mux 输入越多，选择延迟越大；若某个候选结果来自长进位链，它可能成为 ALU 的关键路径。

ALU 常见标志位包括：

标志位	含义	用途
Zero	result 是否为 0	相等判断、条件选择
Carry	无符号进位或借位证据	多字长加法、无符号比较
Overflow	补码有符号溢出	有符号范围检查
Negative	result 最高位是否为 1	补码符号证据

Zero 可以由所有结果 bit 做 OR 后再取反得到。Negative 通常直接来自结果最高位。Carry 来自加法器最高位外的进位。Overflow 则要看有符号加减法中符号是否出现不可能的变化。标志位不是额外装饰，而是后续控制的重要输入。例如比较 A 和 B 是否相等，可以让 ALU 执行 A-B；若结果所有 bit 都是 0，Zero=1，控制器就知道两个输入相等。

三、数据通路让值从旧状态走到新状态

数据通路回答一个问题：值从哪里来，经过哪里，最后保存到哪里。孤立的寄存器和 ALU 还不能完成连续动作。必须把寄存器阵列、选择器、ALU、内部连线和写回路接成一个可控制的闭环。

一个寄存器只能保存一个值。若硬件需要同时保存多个中间值，就需要寄存器阵列。寄存器阵列可以理解为一组编号寄存器，加上读写选择电路。

端口	作用	类型
读地址 A/B	选择两个源寄存器	输入
读数据 A/B	输出两个源值	输出
写地址	选择目标寄存器	输入
写数据	提供要保存的新值	输入
写使能	决定是否在时钟沿写入	输入

读地址进入译码和选择网络，决定哪两个寄存器的值出现在读数据端口。写地址进入译码器，写使能有效时，只允许被选中的那个寄存器在时钟沿接收写数据。这样，一组寄存器就能被编号访问，而不是每个寄存器单独拉一套控制线。

ALU 的两个输入不一定总来自同一处。一个输入可能来自寄存器阵列，也可能来自 PC、计数器或固定 0；另一个输入可能来自寄存器、常量或扩展后的字段。因此 ALU 输入前通常有 mux。

```

alu_a = mux(a_sel, reg_a, counter_value, zero)
alu_b = mux(b_sel, reg_b, constant, offset_value)

```

以寄存器 R1 和 R2 相加、结果写入 R3 为例。数据通路需要让读地址 A 选择 R1，读地址 B 选择 R2；a_sel 选择 reg_a，b_sel 选择 reg_b；ALU op 选择 ADD。此时 ALU 输出就是 R1+R2。

写回数据也不一定只来自 ALU。它可能来自 ALU，也可能来自存储器读口、外部输入或 PC+4。写回前也需要 mux。

```

write_data = mux(wb_sel, alu_result, memory_data, input_data)

```


在 R1+R2 写入 R3 的动作中, wb_sel 选择 alu_result, 写地址选择 R3, reg_write 为 1。等时钟沿到来, R3 才真正保存结果。若 reg_write 为 0, 即使 ALU 算出了结果, 寄存器阵列也不会改变。

当多个部件都可能把值送到同一个目的地时, 不能让它们同时强行驱动同一条线。要么用 mux 明确选择一路, 要么用总线协议规定同一时刻只有一个驱动者。ALU 输出、存储器输出和外部输入如果直接接在一起, 一旦两个源同时输出不同电平, 就会形成冲突。mux 让这件事变成明确选择: 当前周期只允许一路成为 write_data。

一个最小数据通路闭环可以写成:

```
register file -> mux -> ALU -> mux -> register file
```

它能在一个周期内读取旧值、选择输入、完成组合计算, 并在时钟沿保存结果。这个闭环已经能表达很多动作, 但它还不会自己决定“当前周期做什么”。哪些地址送入寄存器阵列, ALU 做加法还是减法, 写回选择哪一路, 写使能是否打开, 都需要控制器统一产生。

四、控制器输出一组一致的控制线

数据通路提供很多可能路径, 但同一周期不能所有路径都随意打开。控制器的任务是根据当前状态和外部条件, 产生一组控制信号, 让数据通路执行一个明确动作。控制器不是写在纸上的命令列表, 而是一块输出控制电线的硬件。

控制信号通常是一组 0/1 或多 bit 选择值:

```
alu_op
a_sel
b_sel
wb_sel
reg_write
mem_read
mem_write
next_state_sel
```

这些信号直接接到 ALU、mux、寄存器阵列和存储器接口。控制信号给错, 数据通路就会走错。例如 reg_write 本应为 0 却变成 1, 某个寄存器会在时钟沿被意外覆盖; wb_sel 本应选择 ALU, 却选了外部输入, 写回值就会错。

若动作规则简单, 可以用组合逻辑直接从输入生成控制信号。例如输入 mode 为 0 时做加法, 为 1 时做按位 AND:

```
mode = 0 -> alu_op = ADD
mode = 1 -> alu_op = AND
```

硬连线控制速度快, 结构也直接。缺点是规则复杂后, 组合逻辑会变大, 修改一个动作可能影响多条控制线。若一个动作无法在一个周期内完成, 就需要状态机。状态机保存“当前走到哪一步”, 每个状态输出一组控制信号, 并决定下一状态。

状态	控制动作	下一状态
S0	选择源寄存器, 锁存操作数	S1
S1	ALU 计算, 结果进入临时寄存器	S2
S2	选择临时结果, 写回目标寄存器	S0

表中每一行都对应真实硬件动作。S0 打开读路径和临时寄存器写使能; S1 设置 ALU 输入和 op, 并保存 ALU 结果; S2 打开写回路路径和目标寄存器写使能。状态机把复杂动作分成多个清楚边界, 避免把所有组合逻辑放入一个过长周期。

五、用控制 trace 走读一次动作

复杂动作可以拆成微操作：读、选择、计算、保存、更新状态。本节所称微操作，指一个周期内同时打开的一组硬件控制信号。以下以 R1 和 R2 相加、结果写入 R3 为例。

```
S0: read_sel_a=R1, read_sel_b=R2
    A_write=1, B_write=1
    reg_write=0, ALU_out_write=0

S1: a_sel=A, b_sel=B, alu_op=ADD
    ALU_out_write=1
    reg_write=0, A_write=0, B_write=0

S2: wb_sel=ALU_out, write_sel=R3
    reg_write=1
    A_write=0, B_write=0, ALU_out_write=0
```

第一步 S0 需要落实为具体控制信号。read_sel_a 选中 R1 后，寄存器阵列的第一个读端口会把 R1 当前保存的 bit 组合驱动到读数据线上；read_sel_b 选中 R2 后，第二个读端口同理输出 R2。等这些组合路径稳定后，时钟沿到来，A_write 和 B_write 让 A、B 锁存器保存这两个值。这个周期里 reg_write 必须为 0，因为此时写回 mux 上的值并不代表最终结果。

第二步 S1 负责 ALU 计算。A、B 锁存器的输出作为 ALU 输入，alu_op=ADD 让 ALU 内部选择加法路径。加法电路不是瞬间得到稳定结果，低位进位会影响高位，输出线上可能经历一小段变化过程。控制器在这个周期打开 ALU_out_write，让时钟沿把已经稳定的 ALU 输出锁存下来；同时 reg_write 仍然保持 0。

第三步 S2 负责写回。wb_sel 选择 ALU_out 锁存器，write_sel 选择 R3，reg_write 在这个周期置 1。到时钟沿时，寄存器阵列把写数据端口上的值保存到 R3。至此，R1 和 R2 没有被改动，R3 得到 R1+R2 的结果，控制器回到空闲或取下一动作的状态。

该例也说明控制错误的风险。若 S2 的 wb_sel 错选成 B 锁存器，R3 写入的就是 R2，而不是相加结果；若 S1 提前打开 reg_write，R3 可能在 ALU 输出尚未稳定时被覆盖；若 S0 忘记关闭 ALU_out_write，旧的计算边界会被无关读数污染。微操作表的价值在于：每一步都能检查哪些线打开、哪些线关闭、哪个状态元件会在时钟沿改变。

本章的经典读法是 trace：从 bit 解释，到 ALU 候选计算，到数据通路值流，再到控制信号时间表。

- 位级证据：补码取负可写成按位取反再加一；减法可复用加法器执行 $a + (\sim b) + 1$ 。
- 标志证据：Carry 适合无符号边界，Overflow 适合补码有符号边界；不能把两者混成同一个“溢出”。
- ALU 证据：候选计算并行产生，选择器选出一个结果，标志位保存结果证据。
- 数据通路证据：为 $R3 = R1 + R2$ 写读地址、ALU 输入选择、alu_op、写回选择、写地址、写使能。
- 控制证据：每个周期哪些写使能为 1，哪些 mux 选择哪一路，ALU 做什么，都要能列成 trace。

- 检查题：若 ALU 结果已经正确，但写回 mux 选择线接错，寄存器结果会怎样？排查时应看 ALU，还是写回路径和控制器？

经典教材会把控制器和数据通路并排讲：数据通路说明“能走哪些路”，控制器说明“本周期允许哪条路”。两者合起来，才是可执行动作。

最小 CPU 与整机硬件

本部分导读

前面已经有了计算、状态、选择和控制。现在把它们合成 CPU，再继续向外连接主存、总线、设备、中断控制器和启动硬件，并通过 8086、80386 等经典芯片观察真实处理器的演进。

CPU、经典芯片与整机硬件

前面已经具备构造处理器的必要部件：组合逻辑、状态元件、时钟边界、ALU、数据通路和控制器。现在可以把它们合成为 CPU，并进一步连接主存、总线、设备和启动硬件，形成整机。本章不只停留在“最小 CPU”模型，还引入 8086、80386 等经典处理器，说明真实芯片如何在位宽、地址空间、总线接口、保护机制和系统结构上逐步扩展。

CPU 可以视为一组同步硬件闭环：PC 指出下一条编码，取指路径取回指令 bit，译码逻辑生成控制信号，数据通路读取和计算，状态元件在时钟边界提交结果。经典芯片的差异，正是这些闭环在位宽、编码、寻址、控制复杂度和外部接口上的不同取舍。

一、从动作编码到最小 CPU

若希望同一套数据通路在不同时间做不同动作，就需要把“这一次做什么”也表示成 bit。动作编码可以先写成最小形式：

```
opcode | dst | src_a | src_b
```

`opcode` 表示动作类型，例如 ADD、SUB、AND；`dst` 表示写入哪个寄存器；`src_a` 和 `src_b` 表示读取哪两个寄存器。控制器读取这些字段后，把它们译成 ALU op、读地址、写地址、写使能和写回选择。编码本身仍然是硬件输入，不是文字命令。

若有一组动作编码连续放在存储结构中，硬件还需要知道当前取哪一条。这个位置由 PC 保存。PC 本质上也是寄存器，只是它保存的是“下一条动作编码的地址”。

部件	作用
PC	保存下一条编码地址
指令存储器	根据 PC 输出当前编码
控制器	根据编码字段产生控制信号
寄存器阵列	保存通用数据
ALU	做整数和逻辑运算
选择器网络	选择 ALU 输入、写回来源和下一 PC

顺序执行时，下一 PC 可以由加法器产生：

```
pc_next = pc_current + instruction_size
```

如果每条编码占 4 个字节，`instruction_size` 就是 4；如果是可变长度指令，下一 PC 的计算会更复杂。关键事实是：顺序执行不是抽象的“下一行”，而是 PC 这个状态寄存器保存了新的地址。

最小单周期模型可以这样走：

阶段	硬件动作	边界
取指	PC 送入指令存储器，得到编码	组合路径
译码	控制器产生 mux、ALU、写使能信号	组合路径
执行	寄存器阵列读值，ALU 计算	组合路径
写回	结果在时钟沿写入寄存器或 PC	状态边界

用一条普通运算编码走读：当前位置 PC=100，指令存储器在地址 100 处保存的

是：

```
ADD dst=R3, src_a=R1, src_b=R2
```

周期开始时，PC 输出稳定为 100。这个地址进入指令存储器，存储器经过地址译码后把这条编码输出。控制器看到 opcode=ADD，于是把 alu_op 设为 ADD，把寄存器阵列的两个读地址分别设为 R1 和 R2，把写地址设为 R3，把 wb_sel 设为 ALU，把 reg_write 设为 1，同时让下一 PC 选择 PC 加指令长度。

这些控制信号稳定后，寄存器阵列的两个读端口输出 R1 和 R2 当前的值。它们经过 ALU 输入选择器进入 ALU，加法路径产生结果。结果再经过写回选择器，到达寄存器阵列的写数据端口。另一边，PC 加法器准备下一条指令地址，PC 选择器把它送到 PC 的下一值输入。只有到时钟沿，两个状态变化才同时提交：R3 保存 ALU 结果，PC 保存下一条地址。

分支也可以归结为下一 PC 选择。顺序情况下，下一 PC 是 PC 加指令长度；条件分支则由 ALU 比较结果和选择器决定下一 PC 来源：

```
branch_taken = condition_met AND branch_op
next_pc = mux(branch_taken, pc_sequential, branch_target)
```

分支的硬件本质是改变 PC 这个寄存器的下一值。下一周期取到哪条编码，完全由这个新 PC 决定。

二、8086：从芯片接口看早期微处理器

8086 是理解早期 x86 体系的重要起点。它是 16-bit 微处理器，拥有 16-bit 通用寄存器和 16-bit 数据通路特征，同时通过 20-bit 地址能力访问最多 1 MiB 物理地址空间。它的历史意义不只在指令集，还在于展示了 CPU 芯片如何通过外部总线、地址锁存、控制信号和存储器/外设共同构成一台机器。

8086 的寄存器组包括 AX、BX、CX、DX、SP、BP、SI、DI，以及 CS、DS、SS、ES 等段寄存器。段寄存器和偏移地址共同形成物理地址：

```
physical_address = segment * 16 + offset
```

这个规则反映了一个时代的硬件取舍：内部仍以 16-bit 编程模型为主，但外部需要访问超过 64 KiB 的地址空间，于是通过段基址左移和偏移相加得到 20-bit 地址。分段不是高级抽象，而是地址形成硬件的一部分。

8086 的外部总线也值得注意。地址线和数据线存在复用，某些引脚在总线周期的不同阶段承担不同含义。硬件需要地址锁存器在地址有效阶段保存地址，随后这些引脚可以用于数据传输。一次总线访问不是“读内存”三个字，而是分阶段发生的事务：输出地址、锁存地址、给出读写控制、等待存储器或 I/O 响应、传输数据、结束总线周期。

8086 观察点	硬件含义
16-bit 寄存器	数据运算和编程模型以 16-bit 为核心
20-bit 地址形成	通过段寄存器和偏移访问 1 MiB 物理空间
地址/数据复用总线	芯片引脚有限，外部需要锁存和分阶段传输
存储器与 I/O 周期	访问目标由控制信号和地址空间规则区分
BIU/EU 分工	取指队列和执行部件可部分重叠工作

8086 内部常被划分为总线接口单元和执行单元。总线接口单元负责取指、形成物理地址、访问外部总线，并维护预取队列；执行单元负责译码和执行指令。这个分工说明，即使没有现代深流水，CPU 内部也已经在尝试把“取指访问外部世界”和“执行内部运算”分开，以减少外部存储访问延迟对执行的影响。

从本书前面建立的硬件模型看，8086 可以视为一个更复杂的同步系统：PC 在 x86 语境中体现为指令指针和代码段组合；地址形成部件把段和偏移送入加法路径；

总线接口把地址和控制信号变成外部事务；执行部件译码指令并驱动寄存器、ALU 和标志位。8086 不是脱离前面章节的新对象，而是前述部件在真实芯片中的组合。

三、80386：32-bit、保护模式与地址转换

80386 把 x86 推入 32-bit 时代。它扩大了通用寄存器宽度，引入 EAX、EBX、ECX、EDX、ESP、EBP、ESI、EDI 等 32-bit 形式；地址和数据处理能力显著增强，并提供保护模式、分页支持和更完整的内存保护机制。相较 8086，80386 展示的不只是“位宽变大”，而是 CPU 开始承担更复杂的系统级硬件职责。

80386 的一个关键变化是 32-bit 线性地址。程序形成的有效地址先经过分段机制得到线性地址；在启用分页时，线性地址再经过页表转换为物理地址。可以用简化链路表示：

```
effective address -> segmentation -> linear address
linear address    -> paging          -> physical address
```

这条链路说明，地址不再只是“寄存器加偏移后送到总线”。CPU 内部必须有地址形成、权限检查、页表访问、异常触发等硬件机制。若页表项不存在、权限不允许、访问越界，CPU 不是简单得到一个错误数据，而是产生异常事件，控制流转入规定入口。

保护模式把“谁能访问什么”变成硬件可检查的合同。段描述符、特权级、页表权限等机制，使得不同程序或系统组件不能随意访问全部内存或执行全部指令。这些机制后来成为现代操作系统隔离、虚拟内存和系统调用的硬件基础。对本书来说，重点不是展开操作系统，而是看见硬件层面的扩展：CPU 不只执行 ALU 运算，还参与地址翻译、权限判断和异常入口选择。

80386 观察点	硬件含义
32-bit 通用寄存器	数据通路和编程模型扩展到 32-bit
32-bit 线性地址	地址空间显著扩大，地址形成更复杂
保护模式	CPU 硬件检查权限、界限和特权级
分页机制	线性地址可经页表映射到物理地址
异常机制	地址或权限错误进入硬件规定的控制入口

从 8086 到 80386，可以看到 CPU 和整机边界的变化。8086 主要展示了早期微处理器如何通过外部总线接入存储器和 I/O；80386 则展示了处理器如何把内存管理和保护机制纳入芯片内部。后来的处理器继续把 cache、TLB、流水线、分支预测、乱序执行、多核互连等机制纳入芯片，但基本问题没有改变：地址从哪里来，权限如何检查，数据经哪条路径返回，状态在哪个边界提交。

四、主存、总线、I/O 与 DMA

寄存器阵列很快，但容量有限。主存提供大容量状态空间，用地址选择位置，用数据线传递内容，用控制信号说明读还是写。

信号	作用
地址	指出访问哪个存储位置
写数据	写入时提供数据
读数据	读取时返回数据
读写控制	指明本次访问方向
ready/valid	指明请求或响应是否有效

一次读访问可以这样理解：CPU 给出地址和读控制，主存经过内部译码、阵列访问和输出驱动后，把数据放到读数据线上，并用响应信号表示结果有效。一次写访问则是 CPU 同时给出地址、写数据和写控制，主存在合适边界把数据保存到对应位

置。

CPU 内部一个周期可以很短，而主存访问可能慢得多。若每次取指、读数据、写数据都等主存完成，ALU、寄存器阵列和控制器会长时间空转。层级存储的动机，是把最近、最常用的一小部分内容放到更近、更快的位置。cache 可以先理解成靠近 CPU 的小型硬件存储，它通过 tag、data、valid、dirty 等字段完成命中判断、数据返回、替换和写回。

cache 字段	作用
tag	判断当前地址是否对应这一项
data	保存从主存取来的数据块
valid	表示这一项是否有效
dirty	表示这一项是否被改过、尚未写回下层

当 CPU、cache、主存、设备控制器都要通信时，需要互连结构。简单系统可以用共享总线，复杂系统会用分层互连或片上网络。无论形式如何，它至少要解决五个问题：谁发起请求，地址指向哪个目标，数据往哪边走，多个发起者冲突时谁先走，响应怎样回到发起者。

设备控制器通常把自己的控制寄存器映射到地址空间中。CPU 对某些地址读写，实际访问的是设备寄存器，这就是 MMIO。写一个控制寄存器可能启动设备动作，读一个状态寄存器可能得到设备是否完成、是否出错、缓冲区是否可用。设备寄存器虽然像内存一样通过地址访问，但语义由设备控制器定义。

若设备要搬运大量数据，由 CPU 逐字读写会占用大量内部数据通路时间。DMA 控制器可以在获得总线使用权后，直接在设备和主存之间搬运数据。CPU 只需要配置源地址、目标地址、长度和方向，真正的数据搬运由 DMA 硬件完成。DMA 也带来新的边界：CPU 和 DMA 都可能访问主存，互连必须仲裁；cache 可能保存旧副本，系统必须规定 DMA 区域怎样与 cache 保持一致；设备完成后还需要可观察的完成信号。

设备完成工作或状态变化时，需要把事件送回 CPU。中断控制器负责收集多个设备的请求，进行屏蔽、优先级排序和投递。站在硬件角度，中断是一条事件入口：外部设备把请求线送进控制结构，CPU 在规定边界看到这个请求，并转入对应处理路径。

五、整机启动与经典芯片后的演进

通电或复位后，CPU、主存控制器、互连和设备控制器都必须进入规定初始状态。PC 通常被置到固定启动地址，那里连接 ROM、片上存储或其他启动介质。启动路径的第一件事，是让硬件从可预测位置开始取编码。

整机启动不仅是 CPU 复位。时钟源要稳定，复位树要按顺序释放，主存控制器要完成训练或初始化，互连要进入可转发请求的状态，必要设备要处于安全默认值。若这些硬件条件没有满足，即使 CPU 本身能取指，也可能在第一次访问主存或设备时无法继续推进。

经典微处理器的发展可以用若干硬件剖面概括。下表不是处理器史年表，而是说明每一代芯片把哪些硬件问题推到读者面前。

芯片或系列	典型特征	对硬件学习的意义
4004	4-bit 微处理器	微处理器把控制器、寄存器和算术部件集成到单芯片形态
8080/8085	8-bit 微处理器	外部总线、地址空间、存储器和 I/O 开始形成通用微机结构
8086/8088	16-bit x86 起点	段：偏移地址、指令预取、复用总线和外部锁存器体现早期整机取舍

80286	保护模式前驱	分段保护和特权机制开始进入处理器硬件合同
80386	32-bit x86	32-bit 寄存器、线性地址、分页和异常机制把 CPU 推向系统级硬件
80486/Pentium	片上 cache 与更深执行结构	流水线、cache、浮点单元和更复杂控制逻辑成为性能核心

从 8086、80386 向后看，处理器继续沿几条方向扩展。第一，内部执行路径更深，流水线把取指、译码、执行、访存、写回切成多个阶段。第二，存储层级更复杂，片上 cache 和 TLB 成为地址访问性能的关键。第三，控制逻辑更复杂，分支预测、乱序执行和异常恢复要求硬件保存更多内部状态。第四，多核和片上互连让“总线事务”扩展成核间一致性和缓存一致性协议。第五，外设从简单寄存器发展到 PCI、PCIe、USB、SATA 等分层协议，但本质仍然是地址、数据、控制、握手、完成和错误状态。

演进方向	保持不变的硬件问题
流水线	状态边界如何切分，错误路径如何清除
cache/TLB	地址如何命中、缺失、替换和回填
保护与异常	何时拒绝访问，如何进入规定处理入口
DMA 与设备	谁发起事务，谁完成事务，完成如何通知 CPU
多核一致性	多个观察者如何看到受控顺序的内存结果

到这里，本书的硬件链路闭合：电和器件形成可控开关，开关形成门，门形成组合逻辑，反馈和时钟形成状态，寄存器和 ALU 形成数据通路，控制器组织动作，CPU 执行编码，主存、cache、互连、设备和启动路径把它扩展成整机。8086 和 80386 的意义在于提供了两个清晰历史剖面：一个展示早期微处理器如何连接总线和地址空间，一个展示处理器如何承担更复杂的地址转换、保护和异常机制。

整机硬件的标注对象是事务和边界。一次访问不是“读内存”三个字，而是地址、方向、数据、发起者、目标、握手、响应和错误状态组成的一次硬件事务。

- 纸上实验：走读一条 ADD 指令、一条条件分支、一次 cache 命中读、一次 cache 未命中读、一次 MMIO 写设备命令、一次 DMA 完成并发中断。
- 经典芯片证据：8086 用段寄存器和偏移形成 20-bit 物理地址；80386 引入 32-bit 地址、保护模式和分页机制。
- 机器证据：主存、cache、设备寄存器都可以被地址访问，但语义不同；地址译码决定请求落到哪类目标。
- 并发证据：DMA 和 CPU 都能成为总线发起者，必须有仲裁；cache 和 DMA 共享主存时，必须有一致性或同步规则。
- 检查题：CPU 写设备命令寄存器后，能否立刻假设设备动作完成？应当看 MMIO 写完成、设备 done 位、中断，还是 DMA completion？

经典系统教材会把整机看成多层抽象：指令看到寄存器和地址，硬件看到事务，设备看到寄存器和状态机，系统软件最终看到完成事件。能把这些层对应起来，才算从最小 CPU 走到了真正机器。

读完后的检查表

读完本书后，至少应该能回答下面这些问题：

主题	能力要求
比特	能说明为什么计算机用两个稳定区间表示 0 和 1，而不是追求连续电压的精确值。
逻辑门	能从开关路径解释 NOT、AND、OR、NAND 的行为，并理解为什么 NAND 可以构造其他门。
组合逻辑	能把真值表转成门电路，解释半加器、全加器、多路选择器和译码器分别解决什么问题。
时序逻辑	能说明锁存器、触发器、寄存器和时钟边界的关系，知道为什么组合逻辑不能保存历史。
状态机	能把状态、输入、转移条件和输出分开，并说明状态机怎样控制一组硬件动作。
ALU	能把加法、减法、按位运算、比较和标志位放在同一个计算部件里解释。
最小 CPU	能画出 PC、指令存储器、寄存器堆、ALU、数据存储器 and 控制器之间的数据流。
整机硬件	能说明主存、总线、MMIO、DMA、中断控制器和复位启动怎样把最小 CPU 扩展成整机。

最终检查不以“能不能复述章节标题”为准，而以能不能给出机器证据为准。

- 给一个信号节点，能说出它何时被拉高、何时被拉低、何时悬空、何时冲突。
- 给一个组合电路，能列真值表、最长路径、可能毛刺和后级采样边界。
- 给一个状态电路，能指出旧状态、下一状态、时钟沿、建立/保持约束和复位行为。
- 给一个算术结果，能分别按无符号和补码解释，并说明 Carry 与 Overflow 的差别。
- 给一条最小 CPU 指令，能写出控制信号 trace、数据通路 trace 和 PC 下一值。
- 给一次内存或设备访问，能写出地址译码、总线/互连事务、响应返回和完成证据。

如果这些证据能写出来，说明读者已经把“电平 -> 门 -> 组合逻辑 -> 状态 -> 数据通路 -> 控制器 -> CPU -> 整机”的链路连成了一台机器。